

DeepTutor: Towards Agentic Personalized Tutoring

Bingxi Zhao, Jiahao Zhang, Xubin Ren, Zirui Guo, Tianzhe Chu, Yi Ma, Chao Huang✉

The University of Hong Kong

🔗 <https://github.com/HKUDS/DeepTutor>

Education is one of the most promising real-world applications for Large Language Models (LLMs). However, current LLMs rely on static pre-training knowledge and lack adaptation to individual learners, while existing RAG systems fall short in delivering personalized, guided feedback. To bridge this gap, we present DeepTutor, a fully open-source agentic framework that unifies citation-grounded problem tutoring with difficulty-calibrated question generation. A hybrid personalization engine couples static knowledge grounding with dynamic learner memory, continuously adapting each interaction to the student’s evolving needs. The same personalization substrate further extends to adaptive learning workflows, interactive books, and proactive multi-channel tutoring agents. To evaluate personalized tutoring, we introduce TutorBench, an interactive benchmark incorporating customized learner profiles grounded in university-level curricula across five domains. We further propose an LLM-based first-person interactive evaluation protocol that conducts assessments via a profile-driven student simulator. Complementary evaluations on established benchmarks, supported by human-alignment and ablation studies, confirm the framework’s robustness and general utility. Results show that DeepTutor improves personalized metrics by 10.8% on average and strengthens general agentic reasoning across five backbone models by 29.4%.

1. Introduction

Education represents one of the most impactful and rapidly evolving real-world domains for the deployment of large language models. An effective human tutor does far more than supply correct answers. They diagnose misconceptions, scaffold explanations to the learner’s level, and continuously pose targeted practice that turns weaknesses into strengths (Létourneau et al., 2025). While LLMs exhibit impressive fluency in educational dialogues, replicating this tightly integrated set of pedagogical skills within a unified, adaptive system remains an open challenge (Chu et al., 2025).

Existing approaches tackle fragments of this challenge in isolation, as illustrated in Figure 1. On the problem-tutoring side, recent LLM tutoring work has moved beyond direct answer generation by evaluating open-ended pedagogical behavior, diagnosing student reasoning errors, and aligning models toward guided problem solving (Daheim et al., 2024; Dinucu-Jianu et al., 2025; Macina et al., 2025). This makes responses more targeted, but the resulting diagnosis is usually consumed within the same tutoring turn. On the question-generation side, LLM-guided generation methods use planning or self-refinement to control question content, difficulty, and educational objectives (Cheng et al., 2025; Li and Zhang, 2024; Yao et al., 2025). These systems insightfully improve item quality, yet their generation process is typically driven by predefined objectives rather than the student’s recent tutoring traces. Together, these workflows remain task-local: tutoring traces do not condition the next practice item, and practice outcomes do not update future explanations. Both shortcomings share a root cause: current systems lack a fine-grained, evolving model of the learner. At best, they track coarse skill inventories rather than the reasoning traces that reveal *how* a student errs (Corbett

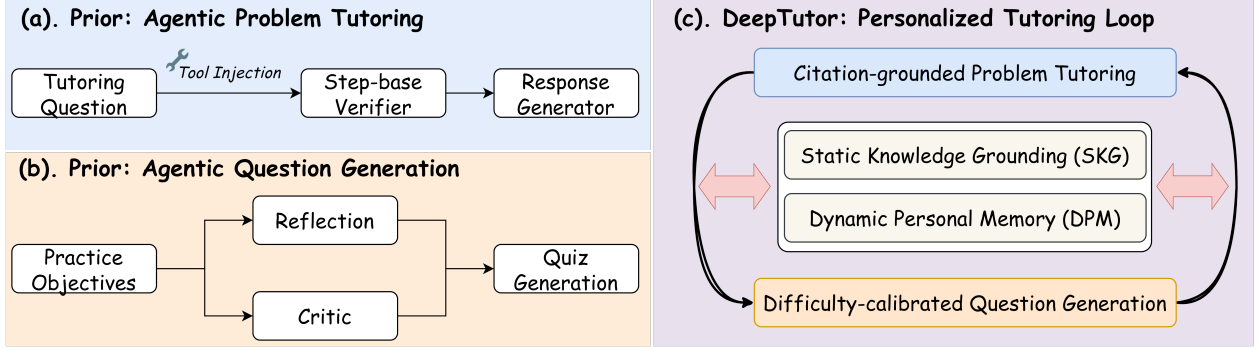


Figure 1 | From isolated education workflows to closed-loop personalized tutoring. (a,b) Prior tutoring and question-generation agents operate task-locally; (c) DeepTutor uses a Hybrid Personalization Engine, combining Static Knowledge Grounding (SKG) and Dynamic Personal Memory (DPM), to form a closed loop between tutoring traces and personalized practice.

and Anderson, 1995; Ghosh et al., 2020; Park et al., 2024; Piech et al., 2015; Wang et al., 2025a; Zhang et al., 2017).

Compounding this system gap is an *evaluation* gap. Most educational benchmarks adopt an instructor-centric perspective, testing whether an LLM follows sound pedagogical principles or produces factually correct answers, while treating the student as a generic receiver (Chu et al., 2025; Kurdi et al., 2020). The few learner-aware efforts represent students with coarse proficiency labels that are disconnected from any specific course material. As a result, whether a tutoring system can truly *adapt* to an individual learner across a multi-turn conversation has remained largely untested.

We argue that addressing both gaps requires a *closed interactive loop*: weaknesses exposed during problem tutoring should directly shape which questions are generated next, while the learner’s performance on those questions should in turn refine the model that informs future explanations (Black and Wiliam, 1998; Shute, 2008). Realizing such a loop demands two ingredients: a structured memory that captures fine-grained interaction histories rather than scalar skill scores, and a unified system whose modules share and continuously update that memory.

To bridge these gaps, we propose **DeepTutor**, which unifies problem tutoring and question generation through a shared personalization engine. Both pipelines are coupled through a *trace forest*, a hierarchical memory whose specialized agents continuously distill interaction traces into an evolving learner profile, creating a closed tutoring cycle in which every interaction personalizes the next. To enable personalized evaluation, we also construct **TutorBench**, a student-centric benchmark built from university-level materials across five broad disciplines. Each entry couples a source-grounded learner profile with diagnosed knowledge gaps and an interactive tutoring task, and a first-person student simulator drives multi-turn dialogue that tests adaptive behavior end to end.

Our main contributions are as follows:

- We introduce DeepTutor, an open-source agentic tutoring framework that closes the loop between citation-grounded problem tutoring and difficulty-calibrated practice generation through shared learner context.
- We design a hybrid personalization engine that combines Static Knowledge Grounding with Dynamic Personal Memory, using a trace forest to distill multi-turn interactions into fine-grained, reusable learner profiles.
- We characterize how the same personalization substrate extends beyond reactive tutoring to broader adaptive learning workflows, interactive books, and proactive multi-channel tutoring agents, while

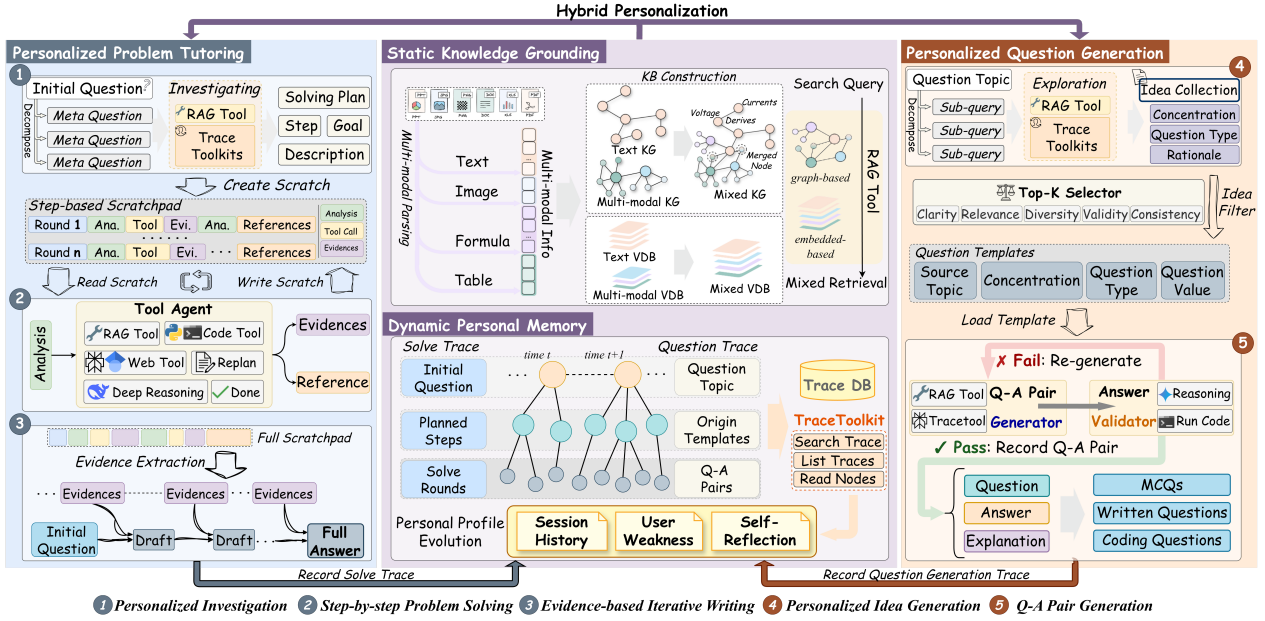


Figure 2 | Overview of DeepTutor. Static Knowledge Grounding supplies course context, Dynamic Personal Memory supplies learner context, and the Hybrid Personalization layer feeds both problem tutoring and question generation. Completed interactions update the trace forest and learner profile, forming a closed tutoring cycle.

separating these system extensions from the quantitatively evaluated tutoring core.

- We construct TutorBench, a student-centric benchmark with first-person interactive evaluation across five university-level disciplines, and show that DeepTutor improves personalized tutoring quality while generalizing across domains and backbone models.

2. DeepTutor Framework

As shown in Figure 2, DeepTutor unifies two tutoring tasks through a shared Hybrid Personalization Engine: *problem tutoring*, which produces citation-grounded, learner-calibrated guidance a for a student question q , and *question generation*, which produces customized practice items for a topic τ targeting the learner’s gaps. The engine couples course knowledge $\mathcal{K}$ with an evolving learner profile $\mathcal{D} = (\mathcal{D}_s, \mathcal{D}_w, \mathcal{D}_r)$, assembling a personalization context C_{mem} before each agent step. Problem tutoring then proceeds through investigation, guided solving, and iterative writing (§2.2), while question generation separates idea selection from verified question–answer–explanation construction (§2.3). Each completed interaction updates the profile and trace history, forming the closed loop summarized in Algorithm 1.

2.1. Hybrid Personalization Engine

Both pipelines require two complementary forms of context: *domain expertise* and *learner awareness*. DeepTutor provides the former through Static Knowledge Grounding and the latter through Dynamic Personal Memory, unified by a profile injection mechanism that assembles role-specific context for each agent.

2.1.1. Static Knowledge Grounding (SKG)

To support real course materials with heterogeneous content, we decompose source documents into atomic content units and preserve modality-aware structure when available (Wang et al., 2024a). These units are then indexed through two complementary structures. A *knowledge graph* $\mathcal{G}$ organizes structural and contextual relations across the extracted content units, while dense encoders project all units into an *embedding index* $\mathcal{B}$ (Guo et al., 2025b). Together, $\mathcal{G}$ and $\mathcal{B}$ form a dual-index retrieval substrate over $\mathcal{K}$.

At query time, graph traversal over $\mathcal{G}$ captures explicit relationships while dense search over $\mathcal{B}$ surfaces semantically similar content. The two candidate sets are fused via reciprocal rank fusion (Cormack et al., 2009), deduplicated, and truncated to a context budget, yielding the domain grounding C_{rag} .

2.1.2. Dynamic Personal Memory (DPM)

While the knowledge base captures *what the course teaches*, the dynamic memory captures *how the individual learner has engaged with it*. Central to this component is the *Trace Forest* $\mathcal{F}$, where each tree records one complete tutoring interaction as a multi-resolution, semantically searchable artifact. Nodes are organized into three levels: *Level 1* stores session-level input and a global summary; *Level 2* captures intermediate planning units from task decomposition; and *Level 3* preserves fine-grained execution records including tool outputs, evidence, and validation outcomes. Every node carries a dense embedding, enabling similarity-based retrieval across the entire forest. This design is inspired by recent agent memory systems that externalize long-term context and organize experience hierarchically across reusable workflows or interaction traces (Packer et al., 2023; Wang et al., 2025b; Zhang et al., 2025, 2026). The concrete node types at each level are task-specific and detailed alongside the respective pipelines (§2.2–§2.3).

Rather than treating $\mathcal{F}$ as a passive store, we expose it through a programmatic *TraceToolkit* with three operations: `SEARCHTRACE` for semantic ANN retrieval across $\mathcal{F}$, `LISTTRACES` for filtered enumeration by time, task type, or topic, and `READNODES` for full-content retrieval together with ancestor paths. This toolkit enables every agent to explore the learner’s history at the resolution it needs.

Profile Construction. Trace Forest provides an analyzable evidence space, but learner profiles are not produced by passively summarizing the latest interaction. For each new trace, three specialized memory agents actively query the TraceToolkit to retrieve related prior sessions, inspect fine-grained nodes when needed, and compare the latest behavior against cross-session patterns before updating $\mathcal{D} = (\mathcal{D}_s, \mathcal{D}_w, \mathcal{D}_r)$. The resulting profile contains three complementary views: $\mathcal{D}_s$ summarizes session history such as topics covered, solving paths, and performance trends; $\mathcal{D}_w$ maintains an evidence-backed inventory of recurring confusions, wrong-answer patterns, and active or resolved knowledge gaps; and $\mathcal{D}_r$ records pedagogical self-reflections that guide future interactions. This process makes profile construction a tool-mediated analysis procedure rather than a single-pass textual summary, grounding personalization in observable trace evidence rather than coarse latent mastery scores.

Before each agent step, the system assembles a personalization context C_{mem} through two channels: (i) *active trace retrieval*, where the TraceToolkit surfaces the most relevant nodes with budget allocated proportionally across levels to balance broad session context with fine-grained precedents; and (ii) *role-specific profile excerpting*, which routes different slices of $\mathcal{D}$ to different agents based on their function (e.g., the planner receives $\mathcal{D}_s$ and $\mathcal{D}_w$; the writer receives $\mathcal{D}_r$; generation agents receive $\mathcal{D}_w$ alongside historical question patterns), reducing irrelevant context for each role. The total token budget is dynamically partitioned between C_{rag} and C_{mem} .

2.2. Personalized Problem Tutoring

Prior tool-augmented agents (Chen et al., 2026; Li et al., 2025d; Wu et al., 2025; Yao et al., 2023) fold planning, retrieval, and composition into a single reasoning loop. This can suffice for short factoid queries but can become brittle in tutoring, where thorough investigation and personalized presentation compete for the same context window. Inspired by *scaffolding theory*, DeepTutor disentangles problem tutoring into three stages: investigate first, guide the solution incrementally, then write adaptively (Wood et al., 1976). These steps correspond to ①–③ in Appendix Algorithm 1.

Stage ①: Personalized Investigation. The planner first runs a lightweight investigation pass before committing to a plan: it decomposes q into meta-questions and gathers evidence across $\mathcal{K}$ (via RAG), the trace forest (via the TraceToolkit), and auxiliary tools when needed. Conditioning jointly on the resulting domain context C_{rag} and learner context C_{mem} , it produces a tutoring plan $\mathcal{P} = \langle s_1, \dots, s_K \rangle$ of concrete, annotated sub-goals. This *investigate-before-plan* design yields sub-goals that are specific to the learner’s actual gaps rather than generically phrased (e.g., “review chain rule in trigonometry” instead of a vague “review calculus”).

Stage ②: Step-by-step Guided Solving. For each sub-goal s_k , the solver first analyzes the target, then acts and records notes using a shared tool suite (Yao et al., 2023), conditioned on both C_{rag} and C_{mem} . Two mechanisms manage the growing reasoning context: *self-notes* distill each step’s outcome into a concise takeaway that later steps reference in lieu of verbose intermediates, and *hierarchical compression* progressively summarizes completed sub-goals into compact digests, freeing context capacity for deeper reasoning on later ones. When the current plan proves inadequate, *adaptive replanning* revises remaining sub-goals while preserving completed work.

Stage ③: Evidence-based Iterative Writing. Rather than concatenating raw tool outputs, the writer extracts structured evidence from the scratchpad and constructs the answer through successive refinement passes, reconciling potentially conflicting findings. The learner context C_{mem} steers both the depth and tone of the explanation to the learner’s *Zone of Proximal Development* (Vygotsky, 1978): *beginners* receive scaffolded step-by-step derivations, while *proficient learners* receive concise summaries foregrounding key insights. Every externally grounded factual claim carries a traceable citation to retrieved evidence, typically from $\mathcal{K}$ and, when enabled, auxiliary web sources.

2.3. Personalized Question Generation

A tutor must also pose the right questions: ones that are meaningful, factually grounded, and calibrated to the individual learner. We propose a two-stage architecture that separates *what to ask* from *how to ask and verify*; these stages correspond to ④–⑤ in Appendix Algorithm 1.

Stage ④: Personalized Idea Generation. Rather than generating questions directly from a bare topic, the idea agent first maps the conceptual landscape around τ through the lens of the individual learner. Conditioning jointly on C_{rag} and C_{mem} , including past mistakes, difficulty trends, and related problems surfaced via the TraceToolkit, the idea agent produces a pool of candidate ideas, each specifying a target concept, question format (MCQ, written-response, or coding), and a personalized rationale grounded in diagnosed gaps. An evaluator-driven filtering and ranking procedure iteratively prunes ill-formed or redundant candidates according to quality dimensions such as clarity, relevance, and diversity, yielding structured question templates $\{\mathcal{T}_i\}$ for the next stage.

Stage ⑤: Critic-Guided Question–Answer–Explanation Generation. For each template $\mathcal{T}_i$, a **generator** produces a question, answer, and explanation triple (q_i, a_i, e_i) , drawing on C_{rag} for factual grounding and C_{mem} for difficulty calibration. A structurally separated **validator** then applies two

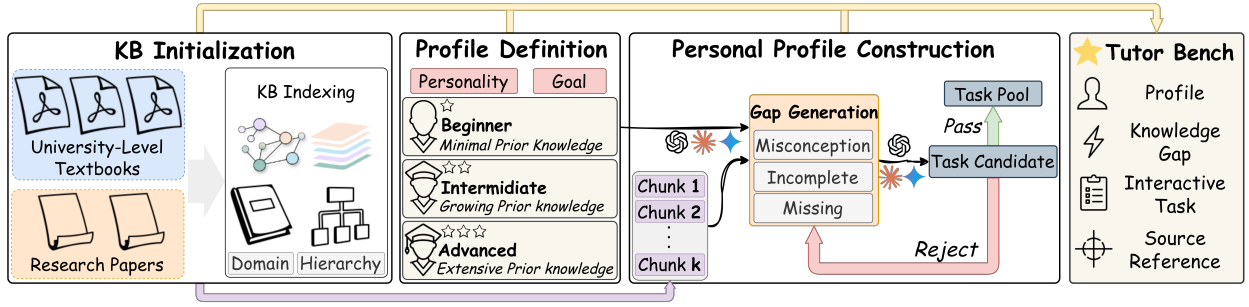


Figure 3 | Construction pipeline of TutorBench. Four stages produce entries each containing a learner profile, grounded knowledge gaps, an interactive task, and source references.

complementary checks: LLM-based verification assesses template alignment, factual correctness, and pedagogical soundness for all items, while computational questions undergo additional sandboxed code execution. This separation is critical: because the validator shares no reasoning chain with the generator, it must independently verify correctness, reducing the risk of self-confirming errors that self-evaluation may miss (Spiliopoulou et al., 2025). Failed pairs receive structured diagnostic feedback and are regenerated until both pedagogical and factual constraints are met.

2.4. Closed-Loop Tutoring Cycle

The components above form a self-reinforcing cycle. After each interaction, the pipeline appends a new trace tree to $\mathcal{F}$ and the three memory agents update $\mathcal{D}$ in parallel, producing a revised profile that immediately informs subsequent sessions. The key property is *bidirectional task coupling*, which operationalizes formative assessment: weaknesses diagnosed during problem tutoring propagate to $\mathcal{D}_w$ and directly shape which questions are generated next; conversely, the learner’s performance on generated questions refines $\mathcal{D}_s$ and $\mathcal{D}_r$, improving future explanations. Because every agent can query the trace forest through the TraceToolkit, this feedback extends beyond profile-level summaries to fine-grained precedents from any prior interaction, enabling increasingly nuanced personalization as the history grows. Appendix B further provides design details of DeepTutor.

3. Extending the Personalization Substrate

The previous section focuses on the quantitatively evaluated core of DEEPTUTOR: citation-grounded problem tutoring, personalized question generation, and learner-memory updates. In practice, however, learning extends beyond a sequence of question-answering turns. Students synthesize information across sources, construct alternative representations of abstract concepts, externalize understanding through writing, and revisit weak concepts over longer time horizons (Graham, 2020; Scardamalia and Bereiter, 2006). If each activity is implemented as an independent tool or interface, the learner model again becomes fragmented. We therefore treat the Hybrid Personalization Engine as a reusable substrate that can support both new agentic capabilities and new learner-facing workspaces. Table 1 summarizes how the main system extensions reuse this substrate at three levels: capability execution, learning surfaces, and long-horizon deployment.

3.1. From Tutoring Workflows to Adaptive Learning

The experimentally evaluated core of DEEPTUTOR personalizes two tutoring capabilities: citation-grounded problem tutoring and difficulty-calibrated question generation. In the implemented system,

Level	Instantiation	Relation to Core Tutoring	Reused Substrate
Capability runtime	Deep Research	First-class agentic capability, parallel to Deep Solve and Question Generation, for open-ended knowledge synthesis.	Query decomposition; KB, web, and paper search; tool execution; report synthesis.
Capability runtime	Visualize / Math Animator	First-class representation capability, parallel to the tutoring capabilities, for diagrams, charts, animations, and demos.	Dialogue context; visual-code generation; review; rendering loop.
Learning surface	Co-Writer Markdown editor	A separate document-editing page that embeds AI writing actions rather than another chat capability.	Markdown workspace; selected spans; optional KB/web grounding; rewrite, expand, shorten, and organize actions.
Curriculum artifact	Book Engine	Composes sources, prior interactions, and capability outputs into an interactive book.	Input snapshot; source exploration; spine and concept graph; typed blocks; quiz progress.
Multi-channel deployment	TutorBot	Extends the DeepTutor capability layer beyond the web interface to proactive channel-based tutoring.	Persona, skills, heartbeat scheduling, channel adapters, and shared capabilities.

Table 1 | System extensions supported by the same personalization substrate.

these capabilities are part of a broader capability runtime rather than isolated features. Deep Research and Visualize extend the same runtime laterally: they are invoked as first-class capabilities alongside Deep Solve and Question Generation, but target different learning needs. Co-Writer extends the system in a different way: it is a Markdown editing workspace with embedded AI actions, so the learner can work on persistent documents while selectively invoking context-aware assistance. This distinction is important because adaptive learning requires both callable agentic capabilities and durable surfaces where learners externalize and refine their understanding.

Adaptive knowledge synthesis. Open-ended learning tasks often require more than answering a single prompt: the learner must identify subtopics, gather evidence, compare sources, and integrate findings into a coherent view (Bereiter, 2002). The Deep Research capability operationalizes this process through a multi-stage agentic pipeline. It first reformulates the learner’s request, decomposes it into research directions, dispatches research agents over course materials, web sources, academic-paper search, and computational tools, and finally synthesizes the evidence into a structured report. Because it shares the same runtime as Deep Solve and Question Generation, the resulting research trajectory can be interpreted as evidence about the learner’s interests, recurring confusions, and emerging knowledge goals.

Adaptive knowledge representation. Textual explanation alone is often insufficient when the target concept is structural, procedural, or geometric. The Visualize and Math Animator capabilities therefore treat representation choice as a pedagogical action rather than a surface formatting step. They analyze the conceptual target and surrounding dialogue, generate executable visual artifacts such as diagrams, charts, animations, or interactive demonstrations, and review the artifact before returning it to the learner. In this view, visualization is selected when the learner state or the content structure suggests that an alternative representation may reduce cognitive load or expose relations hidden in prose (Ainsworth, 2006; Mayer, 2003).

Adaptive knowledge expression. Writing provides a complementary diagnostic surface: students reveal conceptual gaps not only by giving wrong answers, but also by omitting prerequisites, using unstable terminology, or failing to connect claims causally. Co-Writer is therefore implemented as a Markdown editor rather than as another chat workflow. The learner works directly with documents and selected spans, while AI actions can rewrite, expand, shorten, or reorganize text with optional course-material or web grounding. This interface turns writing into a durable learning surface: document revisions and selected passages can become evidence for later tutoring, while assistance remains grounded in the same learner and source context.

Together, these extensions move personalization from local tutoring turns to a broader adaptive learning process. Deep Research and Visualize expand the capability runtime; Co-Writer provides a persistent authoring surface. The common point is that synthesis, representation, and expression can share the same learner-aware substrate instead of introducing separate learner models for each activity.

3.2. Interactive Books through Substrate Reuse

The Book Engine extends the same principle from individual workflows to a curricular artifact. Whereas a tutoring turn produces an answer and a practice request, an interactive book compiles a learner-conditioned collection of sources, prior interactions, notes, and question entries into a persistent learning environment. This design is motivated by the observation that long-horizon learning requires not only local feedback, but also organized paths through concepts, prerequisites, examples, and self-checks. Recent work on AI-augmented textbooks similarly highlights the value of source-faithful personalization, multiple representations, and embedded assessment in transforming static textbooks into adaptive learning experiences (Team et al., 2025).

Input snapshot and proposal. The book workflow begins by freezing a multi-source learning context rather than generating from a bare title. The current implementation accepts user intent, selected chat history, notebook references, knowledge bases, question categories, and question entries as book inputs. An ideation stage turns this input snapshot into a proposal specifying the book’s scope, audience, pedagogical objective, and expected structure. This explicit proposal stage keeps curriculum construction inspectable before expensive generation begins.

Curricular spine and concept graph. After proposal confirmation, a source-exploration stage performs a targeted sweep over the selected materials. A spine synthesizer then constructs the chapter structure and concept graph, making prerequisite relations and topical coverage explicit before page-level compilation. This step reuses the same source-grounding principle as the tutoring core, but changes the unit of organization from a dialogue turn to a curriculum path.

Block-level page compilation. Each page is generated as an ordered sequence of typed blocks rather than a monolithic document. Textual explanation, callouts, quizzes, figures, interactive demonstrations, animations, code, timelines, flash cards, deep-dive links, section summaries, and concept-graph blocks can be selected according to the pedagogical role of the content. This representation makes the book compatible with the adaptive capabilities and surfaces above: research supplies evidence, visualization supplies representational blocks, question generation supplies self-checks, and the writing surface supports learner-authored notes.

Persistent interaction and remediation. The book remains editable and interactive after compilation. Individual blocks can be regenerated or inserted, deep-dive subpages can be spawned from a concept, and quiz attempts update progress records such as visited pages, bookmarks, scores, and weak chapters. Consequently, the book acts less like a static export and more like a persistent learning

workspace. Its interaction logs and weak-chapter signals can provide additional evidence for future learner-profile updates, while the profile can guide subsequent supplementation or remediation.

The Book Engine therefore illustrates a stronger form of substrate reuse: DEEPTUTOR does not merely call tutoring capabilities inside a document viewer; it composes source exploration, concept organization, multimodal block generation, question-based self-checking, and progress tracking into a single adaptive learning artifact.

3.3. Proactive Multi-channel Tutoring

The previous extensions remain primarily learner-initiated: the student chooses when to request research, visualization, writing support, or a book. Long-term learning, however, also depends on timely review and low-friction follow-up. Reactive interfaces require the learner to remember when to return, choose an entry point, and formulate the next request. TUTORBOT addresses this deployment problem by reusing the same tutoring capabilities inside persistent, multi-channel agent instances.

Persistent agent instances. Each TutorBot instance maintains its own workspace, conversation state, persona specification, skill set, and long-term memory while sharing the underlying DEEPTUTOR capability layer. This separation supports specialized agents, such as a mathematics tutor, research assistant, or exam-preparation coach, without duplicating the tutoring runtime. A message adapter converts incoming channel events into the unified context consumed by the orchestrator, so bot-initiated interactions and ordinary tutoring sessions remain compatible.

Capability reuse through skills. TutorBot does not reimplement tutoring behavior. Its skills describe when and how to invoke high-level actions such as grounded explanation, problem solving, question generation, research assistance, visualization, document interaction, or scheduling. Skills therefore act as workflow descriptors over the same tooling and personalization substrate, allowing proactive agents to compose validated capabilities rather than rely on a separate prompt-only interface.

Heartbeat, scheduling, and channels. A heartbeat and scheduling layer allows a bot to periodically decide whether proactive action is pedagogically useful, for example by reminding the learner to revisit an active weakness, generating a short review task after a delay, or summarizing newly added material. Channel adapters route interactions from external communication surfaces into the same backend context. The goal is not to multiply interfaces for their own sake, but to reduce context fragmentation: a learner can interact across devices while the tutor preserves a coherent history and learner model.

We position TutorBot as a deployment mechanism for long-horizon personalized tutoring rather than as an additional intervention validated by the present benchmark. Its effect on retention, engagement, interruption cost, and real learning outcomes requires longitudinal human studies.

4. TutorBench

Existing educational benchmarks predominantly adopt an *instructor-centric* perspective, while treating the student as a generic receiver (Chu et al., 2025). The few learner-aware efforts model students through coarse labels (e.g., “struggling” vs. “proficient”) without grounding profiles in the actual course content (Park et al., 2024; Wang et al., 2025a). Therefore, we construct TutorBench to close this gap: every entry explicitly couples a detailed learner persona with source-grounded knowledge gaps and an interactive tutoring task, as illustrated in Figure 3.

System	Tutoring Quality						Practice Quality						Overall	
	SF	PER	APP	VID	LD	Avg	FIT	GND	DIV	ANS	CC	Avg	OQ	$\Delta\%$
Naive Tutor	3.22	4.24	4.44	3.89	3.99	3.96	<u>3.20</u>	2.46	2.83	3.87	<u>3.12</u>	<u>3.10</u>	3.53	–
CoT Tutor	3.34	4.22	4.47	3.82	4.01	3.97	3.18	<u>2.50</u>	2.79	3.81	3.04	3.06	3.52	-0.28%
Self-Refine Tutor	3.28	4.28	4.61	<u>3.94</u>	<u>4.14</u>	<u>4.05</u>	3.17	2.49	<u>2.88</u>	3.88	2.97	3.08	<u>3.57</u>	+1.13%
ReAct Tutor	<u>3.35</u>	<u>4.37</u>	4.40	3.70	3.96	3.96	3.16	2.47	2.75	<u>3.94</u>	3.07	3.08	3.52	-0.28%
DEEPTUTOR	3.36	4.59	<u>4.56</u>	4.81	4.61	4.39	3.35	2.96	3.44	3.98	3.38	3.42	3.91	+10.76%

Table 2 | Main results of interactive evaluation. Avg is the group mean, OQ means Overall Quality scores across ten metrics. $\Delta\%$ is relative improvement over Naive Tutor. All metrics range from 1 to 5 $\uparrow$. **Bold** represents the best scores, underline represents the second.

Construction Pipeline. University-level textbooks and research papers are first indexed into the dual-representation knowledge base (§2.1.1), from which a domain hierarchy is derived. For each KB, we instantiate three learner profiles using *beginner*, *intermediate*, and *advanced* only as high-level starting points; each profile is further grounded in educational background, learning purpose, and fine-grained per-topic mastery states: *known well*, *partially known*, and *unknown*. For each profile, we sample source pages and generate three types of grounded knowledge gaps: *misconceptions*, *incomplete understanding*, and *missing knowledge* (Shute, 2008; Smith III et al., 1994). Each gap is anchored to specific source pages and includes both a manifestation (how the student would exhibit it) and the correct understanding serving as evaluation reference. During evaluation, all tutor systems receive the same task interface and access to the same source KB. The learner profile and gap manifestations are used to initialize the student simulator, while the correct understandings are reserved for judging and are not exposed to tutor systems as ground-truth answers. Interactive tasks are generated around selected gaps via rejection sampling, which verifies gap coherence, task–gap alignment, and conversational naturalness; the full procedure is provided in Appendix Algorithm 3. All generated profiles and tasks then undergo a final human review, where annotators verify factual correctness, profile plausibility, gap–task consistency, and pedagogical naturalness before inclusion in the benchmark. Full construction details are provided in Appendix C.

Statistics. TutorBench draws on 30 knowledge bases spanning five broad disciplines, covering *humanities*, *sciences*, *engineering*, *business*, and *frontier research*. Based on this coverage, we instantiate three student profiles with different levels for each knowledge base, yielding 90 student profiles grounded in explicit KB materials. After rejection sampling, we retain exactly three accepted interactive tasks for each profile, resulting in 270 tasks in total. All retained entries are further checked by human reviewers before final release. These tasks cover concept understanding, problem solving, application, and comparison-oriented tasks. Every entry carries three source-grounded knowledge gaps, each anchored to specific pages from the source KB materials.

5. Experiments

We evaluate DeepTutor along five complementary axes: *first-person interactive evaluation* (§5.1), *cross-domain generalization* (§5.2), *human preference alignment* (§5.3), *component-level ablation* (§5.4), and *extended generalization of agentic problem solving* (§5.5). The system extensions in Section 3 demonstrate how the same substrate supports broader adaptive learning workflows, interactive books, and proactive agents. Our quantitative evaluation focuses on the foundational closed-loop tutoring core, where controlled benchmarking is feasible; longitudinal evaluation of the interactive-book and TutorBot layers is left for future work.

5.1. First-Person Interactive Evaluation

The adaptive diagnosis and guided remediation that unfold over multiple turns are often missed by traditional single-turn metrics. Building on rubric-based LLM evaluation in open-ended settings (Macina et al., 2025; Zheng et al., 2023) and simulated-student paradigms for tutoring assessment and training (Dinucu-Jianu et al., 2025; Pan et al., 2025; Scarlatos et al., 2026), we introduce a TutorBench protocol in which an LLM-based student simulator engages each tutoring system, and the resulting transcripts are scored against personalized rubrics. Figure 4 summarizes the loop: a benchmark entry initializes the learner profile, source-grounded gaps, task, and materials; the simulator converts the gaps into first-person beliefs, conducts multi-turn tutoring, and then requests customized practice; the resulting traces are judged with personalized rubrics. This design lets the simulator embody the student’s persona rather than merely narrating errors from an external perspective Scarlatos et al. (2026), while dynamic gap pacing elicits resistance patterns that reflect sustained tutoring interactions Pan et al. (2025).

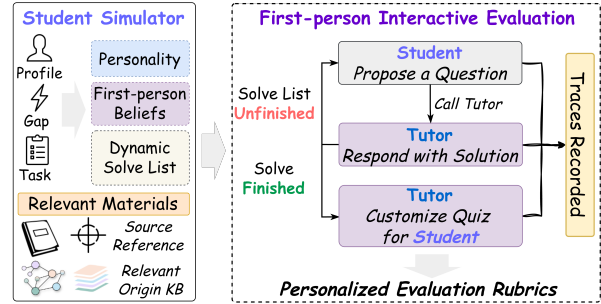


Figure 4 | TutorBench first-person interactive evaluation protocol.

Metrics. To evaluate tutoring from both learner-facing explanation and follow-up practice perspectives, we draw on established work in formative feedback, learner modeling, source-grounded generation, and educational assessment (Anderson et al., 1995; Black and Wiliam, 1998; Corbett and Anderson, 1995; Kurdi et al., 2020; Shute, 2008). This leads to ten complementary dimensions, each scored 1–5 and organized into two groups, rather than a single aggregate quality label. Five **tutoring-side** metrics evaluate the response quality every turn. *Source Faithfulness* (SF) captures factual consistency with the source material and attribution to provided evidence (Maynez et al., 2020; Rashkin et al., 2023). *Personalization* (PER) reflects how specifically the response addresses the learner’s current confusion while remaining aligned with the learner state developed over the session (Corbett and Anderson, 1995; Vygotsky, 1978). *Applicability* (APP) evaluates whether the guidance is concrete and immediately actionable (Hattie and Timperley, 2007; Shute, 2008). *Vividness* (VID) rewards rich, engaging presentation through effective use of multiple representations beyond plain prose (Ainsworth, 2006; Mayer, 2003). *Logical Depth* (LD) assesses the presence of explicit causal chains and intermediate reasoning over bare assertions (Chi et al., 1989; Koedinger et al., 2012). Five **practice-side** metrics score the generated questions after each dialogue. *Fitness* (FIT) measures alignment with session-level diagnosed weaknesses at appropriate difficulty (Lord, 1980; Wood et al., 1976). *Groundedness* (GND) requires the question stem, answer, and explanation to be factually anchored in the source material (Gao et al., 2023; Lewis et al., 2020). *Diversity* (DIV) rewards novelty in angle and cognitive demand (Anderson et al., 2001; Kurdi et al., 2020). *Answer Quality* (ANS) evaluates correctness of the key and plausibility of distractors (Haladyna et al., 2002). *Cross Concept* (CC) tests whether the question meaningfully integrates multiple concepts surfaced throughout the session (Barnett and Ceci, 2002; Chi et al., 1981).

Protocol. Operationally, the evaluation follows three stages: offline entry generation, multi-turn simulation, and LLM-based independent judging. To mitigate the instability of LLM-based evaluation, each transcript is scored three times by a fixed judge and the scores are averaged. For the reported interactive evaluation, we use *Gemini-3-Flash* to power the student simulator as well as all tutor backbones including DeepTutor, while *Claude Sonnet 4.6* serves as the judge with temperature set

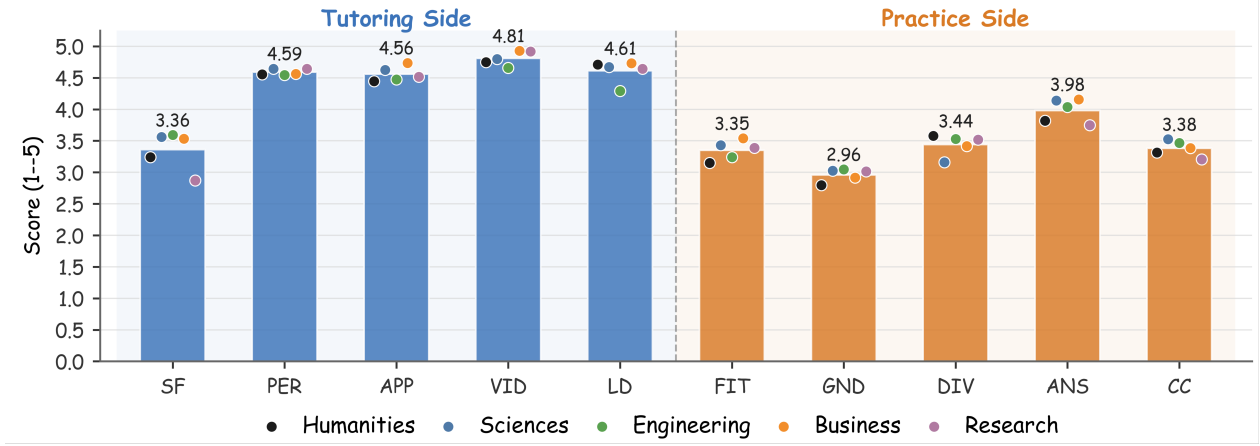


Figure 5 | Cross-domain metric profiles for the first-person interactive simulation. Bars show the average score across the five domains, while colored dots show domain-specific scores.

to zero. All reported interactive results are obtained on the full TutorBench release, covering 270 tasks instantiated from 90 student profiles grounded in 30 knowledge bases. For reproducibility, Appendix D.1 gives the full first-person evaluation details, including entry construction, simulator and judge prompts, scoring protocol, and exact rubric wording.

Baselines. Existing LLM-agent systems for education and open-domain reasoning vary substantially in scope, and several tutoring systems are either unavailable as open-source releases or difficult to reproduce end-to-end in our setting. We therefore design four representative tutor baselines within one unified harness. They share the same RAG tool and backbone with DEEPTUTOR. Among them, *Naive Tutor* uses direct prompting; *CoT Tutor* adds chain-of-thought reasoning (Wei et al., 2022); *Self-Refine Tutor* adds a pedagogical review pass (Madaan et al., 2023); and *ReAct Tutor* implements a *thought-act-observe* reasoning loop (Yao et al., 2023).

Results. Main experimental results are reported in Table 2. The four baselines remain tightly clustered, indicating that adding CoT, self-refinement, or ReAct-style tool use to the same backbone and RAG interface is not sufficient to match DEEPTUTOR’s learner-adaptive behavior. DEEPTUTOR improves overall quality by 10.76% and leads on nearly all metrics. On the tutoring side, the most visible gains appear in *Vividness*, *Personalization*, and *Logical Depth*, matching the method’s separation of investigation, guided solving, and adaptive writing: the system first diagnoses the gap, then builds a causal solution path, and finally presents it at an appropriate level of detail. *Source Faithfulness* and *Applicability* remain competitive, suggesting that personalization does not come at the cost of grounding or actionability. On the practice side, large gains in *Groundedness*, *Diversity*, and *Cross Concept* indicate stronger source anchoring and concept coverage, while smaller gains in *Fitness* and *Answer Quality* suggest more targeted and reliably checked items.

5.2. Cross-Domain Generalization

We further decompose DEEPTUTOR’s first-person interactive performance by metric and discipline in Figure 5, with the complete domain-by-metric table in Appendix D.1.4. This analysis tests whether the investigate–solve–write scaffold depends on one content area, or remains stable across materials with different discourse structures and evidence density. Overall quality varies by only 0.16 points across the five domains, indicating that the gains are not driven by a single discipline. At the metric level, the average cross-domain span is 0.36 points, with the largest fluctuation appearing in *Source*

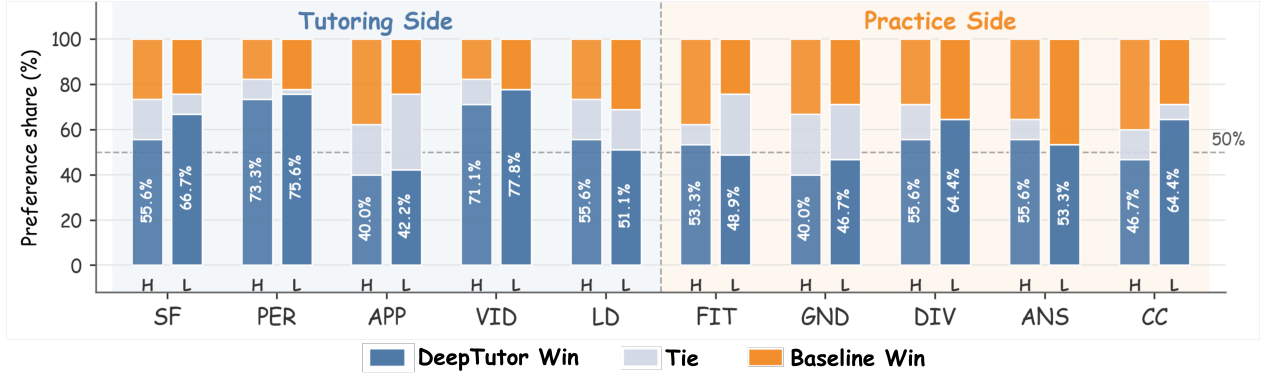


Figure 7 | Human and LLM preference outcomes across ten metrics. H/L denote human-majority and LLM-judge preferences; colors indicate DEEPTUTOR win, tie, and baseline win.

Faithfulness; this suggests that evidence anchoring is more domain-sensitive than learner adaptation. On the tutoring side, *Personalization*, *Applicability*, *Vividness*, and *Logical Depth* are consistently strong, while *Source Faithfulness* is the main relative bottleneck. On the practice side, *Answer Quality* is strongest, whereas *Groundedness* remains the lowest-scoring dimension, suggesting that generating correct practice items is easier than making every question and explanation explicitly traceable to source evidence.

5.3. Human Evaluation of Preference Alignment

To validate whether automated preferences reflect human judgments, we conduct a blind pairwise study on a domain-stratified subset of 45 TutorBench sessions, with nine sessions sampled from each domain. For each selected benchmark entry, human raters compare anonymized DEEPTUTOR and *Naive Tutor (Mock)* outputs generated under the same learner profile, task, source material, student-simulator protocol, and practice-generation request. The comparison packet includes the learner profile, task description, source material, tutoring transcript, and generated practice questions, so raters can judge both tutoring behavior and follow-up practice quality. Raters choose the better system on each metric or mark a tie; disagreements between the two assigned annotators for a session–metric pair are conservatively aggregated as ties. Appendix D.3 documents the annotator instructions, aggregation rules, and additional agreement analyses. Figures 6 and 7 compare automated and human preferences at distribution and metric levels. Both judges assign the largest preference share to DEEPTUTOR across all ten metrics, with the clearest margins on Personalization, Vividness, and Diversity. More contested metrics such as Applicability, Groundedness, and Cross Concept yield more ties or narrower margins, but the trends remain aligned: human and LLM DEEPTUTOR win rates are strongly correlated across the ten metric-level win-rate pairs (Pearson $r = 0.82$, $p = 0.0038$; Spearman $\rho = 0.83$, $p = 0.0027$). This agreement suggests that the LLM judge is not merely favoring DEEPTUTOR globally, but ranks the systems’ relative strengths and weaknesses similarly to human raters under the proposed rubric.

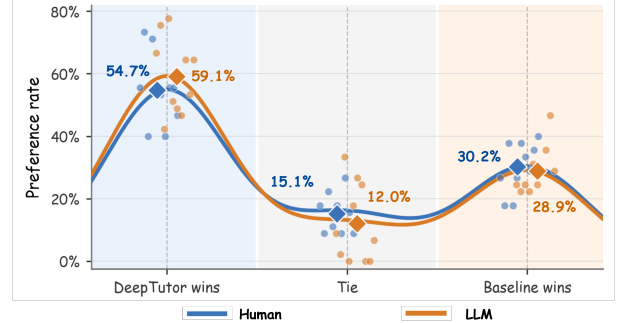


Figure 6 | Distribution-level W/T/L alignment between human raters and the LLM judge.

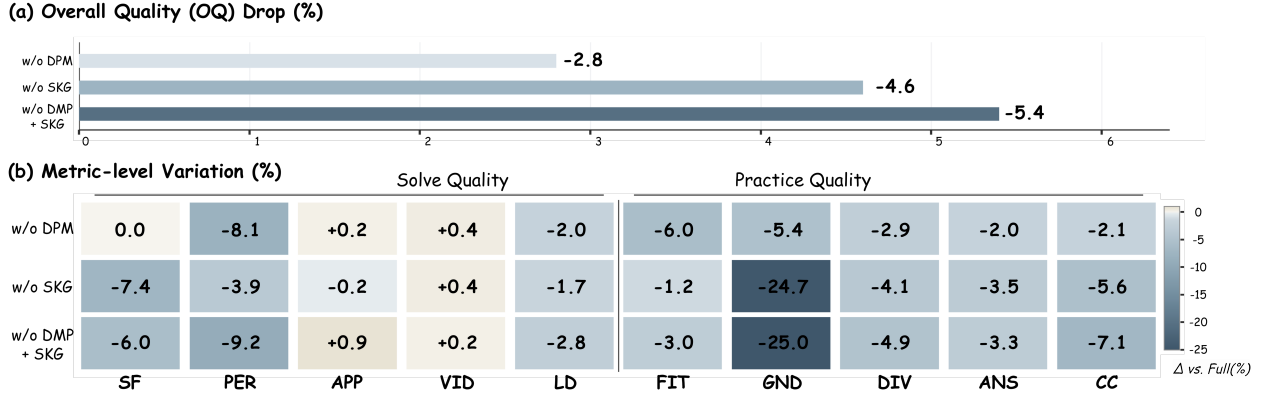


Figure 8 | Component ablation of DEEPTUTOR. Panel (a) shows the overall quality (OQ) drop from removing each module, while panel (b) breaks down the per-metric change relative to the full system.

5.4. Ablation Study

We ablate Static Knowledge Grounding (SKG), Dynamic Personal Memory (DPM), or both modules from the full pipeline to isolate their contributions to personalized tutoring. Figure 8 summarizes both the overall quality degradation and the per-metric changes relative to the full DEEPTUTOR system.

Removing SKG mainly weakens grounding. *Groundedness* drops the most, followed by *Source Faithfulness* and *Cross Concept*, because both explanations and generated questions lose direct access to source evidence for factual anchoring and cross-reference. **Removing DPM** mainly weakens adaptation. *Personalization* and *Fitness* decline most sharply, showing that the learner profile is central to gap-aware explanation and difficulty-calibrated practice; the smaller drop in *Groundedness* indicates that learner context also helps steer retrieval toward relevant evidence. The two ablations therefore affect different parts of the rubric: SKG anchors *what* the tutor says, while DPM shapes *how* it adapts. Removing both modules yields the largest overall degradation, supporting SKG and DPM as complementary mechanisms rather than interchangeable add-ons.

5.5. Extended Generalization of Agentic Problem Solving

Finally, we test whether DeepTutor’s investigate–solve–write scaffold is only useful for personalized tutoring or also transfers to general agentic problem solving. To isolate this effect, we disable the Hybrid Personalization Engine, including SKG and DPM, and evaluate the remaining solver-only pipeline on five public benchmarks: HLE (Phan et al., 2026), GPQA-Diamond (Rein et al., 2024), LiveBench (White et al., 2025), GAIA (Mialon et al., 2024), and AA-LCR (Team, 2025).

For each backbone model m , we summarize performance by averaging the relative gain over its base model across these benchmark groups:

$$\text{Avg } \Delta_m = \frac{1}{|\mathcal{B}|} \sum_{b \in \mathcal{B}} \frac{s_{m,b}^{\text{DEEPTUTOR}} - s_{m,b}^{\text{base}}}{s_{m,b}^{\text{base}}} \times 100\%, \quad (1)$$

where $\mathcal{B}$ denotes the benchmark groups listed above. GAIA L1–L3 scores are reported only as diagnostic breakdowns.

As shown in Table 3, the solver-only pipeline improves all five backbone families, with average relative gains ranging from 25.69% to 32.03%. Because personalization, SKG, and DPM are disabled in

Pass@1	STEM Reasoning			General Agent (GAIA)				Long Ctx	
	HLE*	GPQA-D**	LiveBench***	L1	L2	L3	Overall†	AA-LCR	Avg Δ
Gemini-3-Flash	19.40	81.31	70.00	43.40	40.70	15.38	37.58	63.00	
+DEEPTUTOR	30.80	84.85	96.00	56.60	50.00	23.08	47.88	74.67	+29.24%
Sonnet-4.5	8.40	72.22	64.33	37.74	30.23	7.69	29.09	53.33	
+DEEPTUTOR	14.60	73.23	82.00	50.94	48.84	23.08	45.45	54.00	+32.03%
Qwen-3.5-Plus	16.80	88.38	69.00	44.23	35.71	13.64	33.94	66.00	
+DEEPTUTOR	24.20	87.88	93.00	50.94	53.49	32.00	49.09	69.67	+25.69%
GPT-5-Mini	16.46	80.81	71.00	32.08	30.23	7.69	27.27	68.67	
+DEEPTUTOR	21.20	80.30	93.00	54.72	52.33	26.92	49.09	71.00	+28.51%
Minimax-M2.5	14.00	82.83	59.30	35.29	22.78	12.00	23.64	66.00	
+DEEPTUTOR	19.40	83.33	73.00	52.94	44.30	32.00	42.42	76.40	+31.50%

Table 3 | Results of general agentic problem-solving **Pass@1 scores**. *We use a fixed 500-question subset. **GPQA-Diamond. ***We use the LiveBench reasoning subset. †GAIA scores are reported *per difficulty level (L1–L3)*. Avg Δ is defined in §5.5.

this setting, the gains point to the general value of the investigate–solve–write scaffold rather than learner-specific context alone.

6. Related Work

Personalization and Memory. LLM personalization spans dialogue memory via reflective summarization (Tan et al., 2025) and accumulated user models (Cho et al., 2022; Li et al., 2025a), RAG-style adaptation (Guo et al., 2025b; Li et al., 2025b), workflow induction (Wang et al., 2025b), and hierarchical agent memory (Packer et al., 2023; Zhang et al., 2025, 2026). Classical learner modeling has long been studied through knowledge tracing, from Bayesian KT (Corbett and Anderson, 1995) to neural and memory-based variants (Ghosh et al., 2020; Piech et al., 2015; Zhang et al., 2017). Building on both lines, we propose a tree-structured dynamic memory tailored to tutoring, where interaction traces remain searchable across multiple resolutions.

AI for Education. LLM-based works pursue conversational tutoring with student modeling (Park et al., 2024), multi-agent pedagogical architectures (Wang et al., 2025a), open-source platforms (Hajji et al., 2026), conversational learning diagnosis (Yao et al., 2026), KT-augmented recommendation (Li et al., 2025c), and pedagogically aligned tutor optimization (Dinucu-Jianu et al., 2025). On the evaluation side, benchmarks such as MathTutorBench (Macina et al., 2025) measure open-ended pedagogical quality, but adaptive tutoring grounded in individualized materials remains underexplored. We unify explanation and question generation through a shared learner model and evaluate the closed loop from a student-centric perspective. Appendix A further extends our related-work discussion.

7. Conclusion

We presented DEEPTUTOR, an agentic personalized tutoring framework that closes the loop between citation-grounded tutoring, difficulty-calibrated question generation, and the Hybrid Personalization Engine. We also introduced TUTORBENCH, a student-centric benchmark with source-grounded learner profiles and a first-person simulator for interactive evaluation. Across TutorBench, DEEPTUTOR improves overall quality by 10.8%, remains stable across five domains, and shows preference trends

aligned between human raters and the LLM judge. Beyond tutoring, the solver-only transfer study yields a 29.4% average gain across five backbone families, while ablations show that knowledge grounding and learner memory complement each other in anchoring what the tutor says and shaping how it adapts. The same substrate further supports adaptive learning workflows, interactive books, and proactive multi-channel tutoring agents, suggesting a path from benchmarked tutoring pipelines to long-horizon personalized learning systems. Together, these results provide a practical foundation for closed-loop personalized tutoring agents.

Limitations

Following recent work, our interactive evaluation relies on LLM-powered student simulators and rubric-based assessors, and therefore inherits the usual gap between controlled simulation and real learner behavior. A large-scale validation with human students could be further considered. Like most agentic systems, the multi-stage pipeline trades additional inference cost for stronger controllability and personalization. Moreover, the paradigm of TutorBench could be scaled up to finer-grained courses, longer curricular trajectories, and larger learner populations. The system extensions described in Section 3, including the Book Engine and TutorBot, are presented as architectural instantiations; their effects on retention, engagement, interruption cost, and real learner outcomes require longitudinal human studies.

Broader impacts. Personalized tutoring systems may broaden access to adaptive educational support, especially when high-quality human tutoring is scarce. At the same time, any LLM-based tutor can inherit general LLM risks such as hallucinated facts, overconfident explanations, or subtle reasoning errors. Real-world use should therefore treat generated tutoring content as assistive rather than authoritative, preserve source-grounded verification, and encourage students and educators to check important claims against trusted course materials or human instructors.

References

- S. Ainsworth. DeFT: A conceptual framework for considering learning with multiple representations. *Learning and Instruction*, 16(3):183–198, 2006. doi: 10.1016/j.learninstruc.2006.03.001. URL <https://www.sciencedirect.com/science/article/pii/S0959475206000259>.
- J. R. Anderson, A. T. Corbett, K. R. Koedinger, and R. Pelletier. Cognitive tutors: Lessons learned. *The Journal of the Learning Sciences*, 4(2):167–207, 1995. doi: 10.1207/s15327809jls0402_2. URL https://doi.org/10.1207/s15327809jls0402_2.
- L. W. Anderson, D. R. Krathwohl, P. W. Airasian, K. A. Cruikshank, R. E. Mayer, P. R. Pintrich, J. Rath, and M. C. Wittrock. *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom’s Taxonomy of Educational Objectives*. Longman, 2001. URL <https://www.loc.gov/item/00063423/>.
- S. M. Barnett and S. J. Ceci. When and where do we apply what we learn? a taxonomy for far transfer. *Psychological Bulletin*, 128(4):612–637, 2002. doi: 10.1037/0033-2909.128.4.612. URL <https://pubmed.ncbi.nlm.nih.gov/12081085/>.
- C. Bereiter. *Education and Mind in the Knowledge Age*. Lawrence Erlbaum Associates, Mahwah, NJ, 2002.
- P. Black and D. Wiliam. Assessment and classroom learning. *Assessment in Education: Principles, Policy & Practice*, 5(1):7–74, 1998. doi: 10.1080/0969595980050102. URL <https://doi.org/10.1080/0969595980050102>.

- G. Chen, Z. Qiao, X. Chen, D. Yu, H. Xu, X. Zhao, R. Song, W. Yin, H. Yin, L. Zhang, K. Li, M. Liao, Y. Jiang, P. Xie, F. Huang, and J. Zhou. Iterresearch: Rethinking long-horizon agents with interaction scaling. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=qQ5MZ5Mx7p>.
- Q. Chen, L. Qin, J. Liu, D. Peng, J. Guan, P. Wang, M. Hu, Y. Zhou, T. Gao, and W. Che. Towards reasoning era: A survey of long chain-of-thought for reasoning large language models. *arXiv preprint arXiv:2503.09567*, 2025. URL <https://arxiv.org/abs/2503.09567>.
- C. Cheng, Z. Huang, G. Zhao, Y. Guo, X. Lin, J. Wu, X. Li, and S. Wang. From objectives to questions: A planning-based framework for educational mathematical question generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12836–12856, 2025. doi: 10.18653/v1/2025.acl-long.628. URL <https://aclanthology.org/2025.acl-long.628/>.
- M. T. H. Chi, P. J. Feltovich, and R. Glaser. Categorization and representation of physics problems by experts and novices. *Cognitive Science*, 5(2):121–152, 1981. doi: 10.1207/s15516709cog0502_2. URL <https://www.sciencedirect.com/science/article/pii/S0364021381800298>.
- M. T. H. Chi, M. Bassok, M. W. Lewis, P. Reimann, and R. Glaser. Self-explanations: How students study and use examples in learning to solve problems. *Cognitive Science*, 13(2):145–182, 1989. doi: 10.1207/s15516709cog1302_1. URL <https://www.sciencedirect.com/science/article/pii/0364021389900025>.
- I. Cho, D. Wang, R. Takahashi, and H. Saito. A personalized dialogue generator with implicit user persona detection. In *Proceedings of the 29th International Conference on Computational Linguistics*, pages 367–377, 2022. URL <https://aclanthology.org/2022.coling-1.29/>.
- Z. Chu, S. Wang, J. Xie, T. Zhu, Y. Yan, J. Ye, A. Zhong, X. Hu, J. Liang, P. S. Yu, and Q. Wen. LLM agents for education: Advances and applications. In C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 13782–13810, Suzhou, China, Nov. 2025. Association for Computational Linguistics. ISBN 979-8-89176-335-7. doi: 10.18653/v1/2025.findings-emnlp.743. URL <https://aclanthology.org/2025.findings-emnlp.743/>.
- A. T. Corbett and J. R. Anderson. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4(4):253–278, 1995. doi: 10.1007/BF01099821. URL <https://doi.org/10.1007/BF01099821>.
- G. V. Cormack, C. L. A. Clarke, and S. Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009. doi: 10.1145/1571941.1572114. URL <https://doi.org/10.1145/1571941.1572114>.
- N. Daheim, J. Macina, M. Kapur, I. Gurevych, and M. Sachan. Stepwise verification and remediation of student reasoning errors with large language model tutors. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 8386–8411, 2024. doi: 10.18653/v1/2024.emnlp-main.478. URL <https://aclanthology.org/2024.emnlp-main.478/>.
- D. Dinucu-Jianu, J. Macina, N. Daheim, I. Hakimi, I. Gurevych, and M. Sachan. From problem-solving to teaching problem-solving: Aligning LLMs with pedagogy using reinforcement learning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 272–292, 2025. doi: 10.18653/v1/2025.emnlp-main.15. URL <https://aclanthology.org/2025.emnlp-main.15/>.

- E. Frankford, C. Sauerwein, P. Bassner, S. Krusche, and R. Breu. AI-tutoring in software engineering education. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering Education and Training*, pages 309–319, 2024. doi: 10.1145/3639474.3640061. URL <https://doi.org/10.1145/3639474.3640061>.
- T. Gao, H. Yen, J. Yu, and D. Chen. Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6465–6488, 2023. doi: 10.18653/v1/2023.emnlp-main.398. URL <https://aclanthology.org/2023.emnlp-main.398/>.
- A. Ghosh, N. Heffernan, and A. S. Lan. Context-aware attentive knowledge tracing. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2330–2339, 2020. doi: 10.1145/3394486.3403282. URL <https://doi.org/10.1145/3394486.3403282>.
- S. Graham. The sciences of reading and writing must become more fully integrated. *Reading Research Quarterly*, 55:S57–S76, 2020.
- D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645:633–638, 2025a. doi: 10.1038/s41586-025-09422-z. URL <https://doi.org/10.1038/s41586-025-09422-z>.
- Z. Guo, X. Ren, L. Xu, J. Zhang, and C. Huang. Rag-anything: All-in-one rag framework, 2025b. URL <https://arxiv.org/abs/2510.12323>.
- M. E. Hajji, T. A. Baha, A. Dakir, H. Fadili, and Y. Es-Saady. Open tutorAI: An open-source platform for personalized and immersive learning with generative AI. *arXiv preprint arXiv:2602.07176*, 2026. URL <https://arxiv.org/abs/2602.07176>.
- T. M. Haladyna, S. M. Downing, and M. C. Rodriguez. A review of multiple-choice item-writing guidelines for classroom assessment. *Applied Measurement in Education*, 15(3):309–333, 2002. doi: 10.1207/S15324818AME1503_5. URL https://www.tandfonline.com/doi/abs/10.1207/S15324818AME1503_5.
- J. Hattie and H. Timperley. The power of feedback. *Review of Educational Research*, 77(1):81–112, 2007. doi: 10.3102/003465430298487. URL <https://journals.sagepub.com/doi/10.3102/003465430298487>.
- Y. Hu, S. Liu, Y. Yue, G. Zhang, B. Liu, F. Zhu, J. Lin, H. Guo, S. Dou, Z. Xi, S. Jin, J. Tan, Y. Yin, J. Liu, Z. Zhang, Z. Sun, Y. Zhu, H. Sun, B. Peng, Z. Cheng, X. Fan, J. Guo, X. Yu, Z. Zhou, Z. Hu, J. Huo, J. Wang, Y. Niu, Y. Wang, Z. Yin, X. Hu, Y. Liao, Q. Li, K. Wang, W. Zhou, Y. Liu, D. Cheng, Q. Zhang, T. Gui, S. Pan, Y. Zhang, P. Torr, Z. Dou, J.-R. Wen, X. Huang, Y.-G. Jiang, and S. Yan. Memory in the age of AI agents. *arXiv preprint arXiv:2512.13564*, 2025. URL <https://arxiv.org/abs/2512.13564>.
- N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=chfJJYC3iL>.
- M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. OpenVLA: An open-source vision-language-action model. In *8th Annual Conference on Robot Learning*, 2024. URL <https://openreview.net/forum?id=ZMnD6QZAE6>.

- K. R. Koedinger, A. T. Corbett, and C. Perfetti. The knowledge-learning-instruction framework: Bridging the science-practice chasm to enhance robust student learning. *Cognitive Science*, 36(5): 757–798, 2012. doi: 10.1111/j.1551-6709.2012.01245.x. URL <https://pubmed.ncbi.nlm.nih.gov/22486653/>.
- G. Kurdi, J. Leo, B. Parsia, U. Sattler, and S. Al-Emari. A systematic review of automatic question generation for educational purposes. *International Journal of Artificial Intelligence in Education*, 30(1):121–204, 2020. doi: 10.1007/s40593-019-00186-y. URL <https://doi.org/10.1007/s40593-019-00186-y>.
- A. Létourneau, M. Deslandes Martineau, P. Charland, A.-J. Karran, J. Boasen, and P. M. Léger. A systematic review of AI-driven intelligent tutoring systems (ITS) in K-12 education. *npj Science of Learning*, 10(1):29, 2025. doi: 10.1038/s41539-025-00320-7. URL <https://www.nature.com/articles/s41539-025-00320-7>.
- P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020. URL https://papers.neurips.cc/paper_files/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html.
- H. Li, C. Yang, A. Zhang, Y. Deng, X. Wang, and T.-S. Chua. Hello again! LLM-powered personalized agent for long-term dialogue. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5259–5276, 2025a. doi: 10.18653/v1/2025.naacl-long.272. URL <https://aclanthology.org/2025.naacl-long.272/>.
- K. Li and Y. Zhang. Planning first, question second: An LLM-guided method for controllable question generation. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 4715–4729, 2024. doi: 10.18653/v1/2024.findings-acl.280. URL <https://aclanthology.org/2024.findings-acl.280/>.
- X. Li, P. Jia, D. Xu, Y. Wen, Y. Zhang, W. Zhang, W. Wang, Y. Wang, Z. Du, X. Li, Y. Liu, H. Guo, R. Tang, and X. Zhao. A survey of personalization: From RAG to agent. *arXiv preprint arXiv:2504.10147*, 2025b. URL <https://arxiv.org/abs/2504.10147>.
- Z. Li, J. Wang, W. Gu, V. Yazdanpanah, L. Shi, A. I. Cristea, S. Kiden, and S. Stein. TutorLLM: Customizing learning recommendations with knowledge tracing and retrieval-augmented generation. In *Human-Computer Interaction – INTERACT 2025*, Lecture Notes in Computer Science, pages 137–146. Springer Nature Switzerland, 2025c. doi: 10.1007/978-3-032-05005-2_8. URL https://link.springer.com/chapter/10.1007/978-3-032-05005-2_8. First online: 2025-09-09.
- Z. Li, H. Zhang, S. Han, S. Liu, J. Xie, Y. Zhang, Y. Choi, J. Zou, and P. Lu. In-the-flow agentic system optimization for effective planning and tool use. In *NeurIPS 2025 Workshop on Efficient Reasoning*, 2025d. URL <https://openreview.net/forum?id=XHkX4DbZUv>.
- F. M. Lord. *Applications of Item Response Theory to Practical Testing Problems*. Lawrence Erlbaum Associates, 1980. URL https://www.ets.org/research/policy_research_reports/publications/book/1980/jexj.html.
- J. Macina, N. Daheim, I. Hakimi, M. Kapur, I. Gurevych, and M. Sachan. MathTutorBench: A benchmark for measuring open-ended pedagogical capabilities of LLM tutors. In *Proceedings of the*

- 2025 Conference on Empirical Methods in Natural Language Processing, pages 204–221, 2025. doi: 10.18653/v1/2025.emnlp-main.11. URL <https://aclanthology.org/2025.emnlp-main.11/>.
- A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhume, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36:46534–46594, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html.
- R. E. Mayer. The promise of multimedia learning: Using the same instructional design methods across different media. *Learning and Instruction*, 13(2):125–139, 2003. doi: 10.1016/S0959-4752(02)0016-6. URL <https://www.sciencedirect.com/science/article/pii/S095947520200166>.
- J. Maynez, S. Narayan, B. Bohnet, and R. McDonald. On faithfulness and factuality in abstractive summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 1906–1919, 2020. doi: 10.18653/v1/2020.acl-main.173. URL <https://aclanthology.org/2020.acl-main.173/>.
- G. Mialon, C. Fourrier, T. Wolf, Y. LeCun, and T. Scialom. GAIA: a benchmark for general AI assistants. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=fiBxvAhvS3>.
- C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. URL <https://arxiv.org/abs/2310.08560>.
- S. Pan, R. Schmucker, B. Garcia Bulle Bueno, S. A. Llanes, F. Albo Alarcón, H. Zhu, A. Teo, and M. Xia. TutorUp: What if your students were simulated? training tutors to address engagement challenges in online learning. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pages 1–18, 2025. doi: 10.1145/3706598.3713589. URL <https://doi.org/10.1145/3706598.3713589>.
- M. Park, S. Kim, S. Lee, S. Kwon, and K. Kim. Empowering personalized learning through a conversation-based tutoring system with student modeling. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, pages 1–10, 2024. doi: 10.1145/3613905.3651122. URL <https://doi.org/10.1145/3613905.3651122>.
- L. Phan, A. Gatti, N. Li, A. Khoja, R. Kim, R. Ren, J. Hausenloy, O. Zhang, M. Mazeika, D. Hendrycks, Z. Han, J. Hu, H. Zhang, C. B. C. Zhang, M. Shaaban, J. Ling, S. Shi, M. Choi, A. Agrawal, A. Chopra, A. Nattanmai, G. McKellips, A. Cheraku, A. Suhail, E. Luo, M. Deng, J. Luo, A. Zhang, K. Jindel, J. Paek, K. Halevy, A. Baranov, M. Liu, A. Avadhanam, D. Zhang, V. Cheng, B. Ma, E. Fu, L. Do, J. Lass, H. Yang, S. Sunkari, V. Bharath, V. Ai, J. Leung, R. Agrawal, A. Zhou, K. Chen, T. Kalpathi, Z. Xu, G. Wang, T. Xiao, E. Maung, S. Lee, R. Yang, R. Yue, B. Zhao, J. Yoon, X. Sun, A. Singh, C. Peng, T. Osbey, T. Wang, D. Echeazu, T. Wu, S. Patel, V. Kulkarni, V. Sundarapandian, A. Le, Z. Nasim, S. Yalam, R. Kasamsetty, S. Samal, D. Sun, N. Shah, A. Saha, A. Zhang, L. Nguyen, L. Nagumalli, K. Wang, A. Wu, A. Telluri, S. Yue, A. Wang, D. Dodonov, T. Nguyen, J. Lee, D. Anderson, M. Doroshenko, A. C. Stokes, M. Mahmood, O. Pokutnyi, O. Iskra, J. P. Wang, J.-C. Levin, M. Kazakov, F. Feng, S. Y. Feng, H. Zhao, M. Yu, V. Gangal, C. Zou, Z. Wang, S. Popov, R. Gerbicz, G. Galgon, J. Schmitt, W. Yeadon, Y. Lee, S. Sauers, A. Sanchez, F. Giska, M. Roth, S. Riis, S. Utpala, N. Burns, G. M. Goshu, M. M. Naiya, C. Agu, Z. Giboney, A. Cheatom, F. Fournier-Facio, S.-J. Crowson, L. Finke,

- Z. Cheng, J. Zampese, R. G. Hoerr, M. Nandor, H. Park, T. Gehringer, J. Cai, B. McCarty, A. C. Garretson, E. Taylor, D. Sileo, Q. Ren, U. Qazi, L. Li, J. Nam, J. B. Wydallis, P. Arkhipov, J. W. L. Shi, A. Bacho, C. G. Willcocks, H. Cao, S. Motwani, E. de Oliveira Santos, J. Veith, E. Vendrow, D. Cojoc, K. Zenitani, J. Robinson, L. Tang, Y. Li, J. Vendrow, N. W. Fraga, V. Kuchkin, A. P. Maksimov, P. Marion, D. Efremov, J. Lynch, K. Liang, A. Mikov, A. Gritsevskiy, J. Guillod, G. Demir, D. Martinez, B. Pageler, K. Zhou, S. Soori, O. Press, H. Tang, P. Rissone, S. R. Green, L. Brüssel, M. Twayana, A. Dieuleveut, J. M. Imperial, A. Prabhu, J. Yang, N. Crispino, A. Rao, D. Zvonkine, G. Loiseau, M. Kalinin, M. Lukas, C. Manolescu, N. Stambaugh, S. Mishra, T. Hogg, C. Bosio, B. P. Coppola, J. Salazar, J. Jin, R. Sayous, S. Ivanov, P. Schwaller, S. Senthilkumar, A. M. Bran, A. Algaba, K. Van den Houte, L. Van Der Sypt, B. Verbeken, D. Noever, A. Kopylov, B. Myklebust, B. Li, L. Schut, E. Zheltonozhskii, Q. Yuan, D. Lim, R. Stanley, T. Yang, J. Maar, J. Wykowski, M. Oller, A. Sahu, C. G. Ardito, Y. Hu, A. G. K. Kamdoun, A. Jin, T. G. Vilchis, Y. Zu, M. Lackner, J. Koppel, G. Sun, D. S. Antonenko, S. Chern, B. Zhao, P. Arsene, J. M. Cavanagh, D. Li, J. Shen, D. Crisostomi, W. Zhang, A. Dehghan, S. Ivanov, D. Perrella, N. Kaparov, A. Zang, I. Sucholutsky, A. Kharlamova, D. Orel, V. Poritski, S. Ben-David, Z. Berger, P. Whitfill, M. Foster, D. Munro, L. Ho, S. Sivarajan, D. B. Hava, A. Kuchkin, D. Holmes, A. Rodriguez-Romero, F. Sommerhage, A. Zhang, R. Moat, K. Schneider, Z. Kazibwe, D. Clarke, D. H. Kim, F. M. Dias, S. Fish, V. Elser, T. Kreiman, V. E. G. Vilchis, I. Klose, U. Anantheswaran, A. Zweiger, K. Rawal, J. Li, J. Nguyen, N. Daans, H. Heidinger, M. Radionov, V. Rozhoň, V. Ginis, C. Stump, N. Cohen, R. Poświata, J. Tkadlec, A. Goldfarb, C. Wang, P. Padlewski, S. Barzowski, K. Montgomery, R. Stendall, J. Tucker-Foltz, J. Stade, T. R. Rogers, T. Goertzen, D. Grabb, A. Shukla, A. Givré, J. A. Ambay, A. Sen, C. for AI Safety, S. AI, and H. C. Consortium. A benchmark of expert-level academic questions to assess ai capabilities. *Nature*, 649(8099):1139–1146, 2026. doi: 10.1038/s41586-025-09962-4. URL <https://doi.org/10.1038/s41586-025-09962-4>.
- C. Piech, J. Bassen, J. Huang, S. Ganguli, M. Sahami, L. Guibas, and J. Sohl-Dickstein. Deep knowledge tracing. In *Advances in Neural Information Processing Systems*, volume 28, pages 505–513, 2015. URL <https://papers.neurips.cc/paper/5654-deep-knowledge-tracing>.
- H. Rashkin, V. Nikolaev, M. Lamm, L. Aroyo, M. Collins, D. Das, S. Petrov, G. S. Tomar, I. Turc, and D. Reitter. Measuring attribution in natural language generation models. *Computational Linguistics*, 49(4):777–840, 2023. doi: 10.1162/coli_a_00486. URL <https://direct.mit.edu/coli/article/49/4/777/116438/Measuring-Attribution-in-Natural-Language>.
- D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=Ti67584b98>.
- M. Scardamalia and C. Bereiter. Knowledge building: Theory, pedagogy, and technology. *The Cambridge Handbook of the Learning Sciences*, pages 97–115, 2006.
- A. Scarlatos, J. Lee, S. Woodhead, and A. Lan. Simulated students in tutoring dialogues: Substance or illusion? *arXiv preprint arXiv:2601.04025*, 2026. URL <https://arxiv.org/abs/2601.04025>.
- T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html.
- V. J. Shute. Focus on formative feedback. *Review of Educational Research*, 78(1):153–189, 2008. doi: 10.3102/0034654307313795. URL <https://doi.org/10.3102/0034654307313795>.

- J. P. Smith III, A. A. diSessa, and J. Roschelle. Misconceptions reconceived: A constructivist analysis of knowledge in transition. *The Journal of the Learning Sciences*, 3(2):115–163, 1994. doi: 10.1207/s15327809jls0302_1. URL https://doi.org/10.1207/s15327809jls0302_1.
- E. Spiliopoulou, R. Fogliato, H. Burnsky, T. Soliman, J. Ma, G. Horwood, and M. Ballesteros. Play favorites: A statistical method to measure self-bias in LLM-as-a-judge. *arXiv preprint arXiv:2508.06709*, 2025. URL <https://arxiv.org/abs/2508.06709>.
- Z. Tan, J. Yan, I.-H. Hsu, R. Han, Z. Wang, L. Le, Y. Song, Y. Chen, H. Palangi, G. Lee, A. R. Iyer, T. Chen, H. Liu, C.-Y. Lee, and T. Pfister. In prospect and retrospect: Reflective memory management for long-term personalized dialogue agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8416–8439, 2025. doi: 10.18653/v1/2025.acl-long.413. URL <https://aclanthology.org/2025.acl-long.413/>.
- A. A. Team. Artificial analysis long context reasoning benchmark(lcr), 2025. URL <https://huggingface.co/datasets/ArtificialAnalysis/AA-LCR>.
- L. Team, A. Martín, A. Globerson, A. Wang, A. Shekhawat, A. Iurchenko, A. Choudhury, A. Hassidim, A. Çakmakli, A. S. Evron, et al. Towards an AI-augmented textbook. *arXiv preprint arXiv:2509.13348*, 2025. doi: 10.48550/arXiv.2509.13348. URL <https://arxiv.org/abs/2509.13348>.
- L. S. Vygotsky. *Mind in society: The development of higher psychological processes*, volume 86. Harvard university press, 1978. ISBN 9780674576292. URL <https://www.hup.harvard.edu/books/9780674576292>.
- B. Wang, C. Xu, X. Zhao, L. Ouyang, F. Wu, Z. Zhao, R. Xu, K. Liu, Y. Qu, F. Shang, B. Zhang, L. Wei, Z. Sui, W. Li, B. Shi, Y. Qiao, D. Lin, and C. He. MinerU: An open-source solution for precise document content extraction. *arXiv preprint arXiv:2409.18839*, 2024a. URL <https://arxiv.org/abs/2409.18839>.
- C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao. YOLOv9: Learning what you want to learn using programmable gradient information. In *Computer Vision – ECCV 2024*, Lecture Notes in Computer Science, pages 1–21. Springer Nature Switzerland, 2024b. doi: 10.1007/978-3-031-72751-1_1. URL https://link.springer.com/chapter/10.1007/978-3-031-72751-1_1. First online: 2024-10-26.
- T. Wang, Y. Zhan, J. Lian, Z. Hu, N. J. Yuan, Q. Zhang, X. Xie, and H. Xiong. LLM-powered multi-agent framework for goal-oriented learning in intelligent tutoring system. In *Companion Proceedings of the ACM on Web Conference 2025*, pages 510–519, 2025a. doi: 10.1145/3701716.3715244. URL <https://doi.org/10.1145/3701716.3715244>.
- Z. Z. Wang, J. Mao, D. Fried, and G. Neubig. Agent workflow memory. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 63897–63911. PMLR, 2025b. URL <https://proceedings.mlr.press/v267/wang25bx.html>.
- J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022. URL https://papers.neurips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html.

- C. White, S. Dooley, M. Roberts, A. Pal, B. Feuer, S. Jain, R. Shwartz-Ziv, N. Jain, K. Saifullah, S. Dey, Shubh-Agrawal, S. S. Sandha, S. V. Naidu, C. Hegde, Y. LeCun, T. Goldstein, W. Neiswanger, and M. Goldblum. Livebench: A challenging, contamination-limited LLM benchmark. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=sKYHBTaxVa>.
- D. Wood, J. S. Bruner, and G. Ross. The role of tutoring in problem solving. *Journal of Child Psychology and Psychiatry*, 17(2):89–100, 1976. doi: 10.1111/j.1469-7610.1976.tb00381.x. URL <https://doi.org/10.1111/j.1469-7610.1976.tb00381.x>.
- J. Wu, J. Zhu, Y. Liu, M. Xu, and Y. Jin. Agentic reasoning: A streamlined framework for enhancing LLM reasoning with agentic tools. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 28489–28503, 2025. doi: 10.18653/v1/2025.acl-long.1383. URL <https://aclanthology.org/2025.acl-long.1383/>.
- F. Yao, S. Chang, W. Gao, and Q. Liu. Conversational learning diagnosis via reasoning multi-turn interactive learning. *arXiv preprint arXiv:2603.03236*, 2026. URL <https://arxiv.org/abs/2603.03236>.
- S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.
- Z. Yao, A. Parashar, H. Zhou, W. S. Jang, F. Ouyang, Z. Yang, and H. Yu. MCQG-SRefine: Multiple choice question generation and evaluation with iterative self-critique, correction, and comparison feedback. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 10728–10777, 2025. doi: 10.18653/v1/2025.naacl-long.538. URL <https://aclanthology.org/2025.naacl-long.538/>.
- G. Zhang, M. Fu, K. Wang, G. Wan, M. Yu, and S. YAN. G-memory: Tracing hierarchical memory for multi-agent systems. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=mmIap3cVS0>.
- J. Zhang, X. Shi, I. King, and D.-Y. Yeung. Dynamic key-value memory networks for knowledge tracing. In *Proceedings of the 26th International Conference on World Wide Web*, pages 765–774, 2017. doi: 10.1145/3038912.3052580. URL <https://doi.org/10.1145/3038912.3052580>.
- N. Zhang, X. Yang, Z. Tan, W. Deng, and W. Wang. HiMem: Hierarchical long-term memory for LLM long-horizon agents. *arXiv preprint arXiv:2601.06377*, 2026. URL <https://arxiv.org/abs/2601.06377>.
- B. Zhao, L. G. Foo, P. Hu, C. Theobalt, H. Rahmani, and J. Liu. LLM-based agentic reasoning frameworks: A survey from methods to scenarios, 2025. URL <https://arxiv.org/abs/2508.17692>.
- L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36:46595–46623, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html.

Appendices

A	Extended Related Work	25
A.1	Tool-Augmented LLM Agents	25
A.2	LLM Personalization and Memory	25
A.3	Knowledge Tracing and Intelligent Tutoring Systems	25
A.4	LLM-Based Educational Agents	26
B	Implementation Details of DeepTutor System	26
B.1	Closed-Loop Personalized Tutoring	26
B.2	TutorBot Autonomous Agent Loop	26
C	TutorBench Construction Details	28
C.1	PDF Source Inventory	28
C.2	Task Generation and Rejection Sampling	29
C.3	Trace Forest and Benchmark Runtime Memory	30
D	Evaluation Protocol and Hyperparameters	30
D.1	First-Person Interactive Evaluation Details	30
D.1.4	Cross-Domain Metric Table	34
D.2	Baseline and Variant Definitions	34
D.3	Human Preference Alignment Study	36
D.4	General Problem-Solving Setup	37
D.5	Agent Hyperparameters	38
D.6	Implementation Details	38
D.7	Backbone Models	39

A. Extended Related Work

This appendix expands the literature review in §6 that provides context for the design of an agentic tutoring system, including agentic tool use, memory, learner modeling, and LLM-based educational agents.

A.1. Tool-Augmented LLM Agents

Equipping language models with external tools has progressed from *static* integration to *dynamic* reasoning. Toolformer (Schick et al., 2023) showed that LLMs can learn to insert API calls through self-supervised fine-tuning. ReAct (Yao et al., 2023) introduced a more flexible paradigm by interleaving chain-of-thought reasoning with environment-facing actions, significantly improving multi-step retrieval and decision-making. Subsequent work has expanded planning capabilities in various directions (Li et al., 2025d; Wu et al., 2025; Zhao et al., 2025), while long-horizon agents such as IterResearch (Chen et al., 2026) tackle coherent state maintenance across many reasoning steps. Many existing agentic frameworks are still organized around *bounded* episodes, where a single query triggers a reasoning trajectory that ends once an answer is produced. Tutoring, however, often extends across multiple sessions and benefits from maintaining continuity with an evolving model of the individual learner.

A.2. LLM Personalization and Memory

This stateful requirement connects agentic tool use to a parallel line of work on LLM personalization and memory, which has been pursued along three main axes. *Dialogue-level memory* maintains context across conversation turns through reflective summarization (Tan et al., 2025), accumulated user models (Li et al., 2025a), or implicit persona detection (Cho et al., 2022). *Workflow-level memory* goes beyond individual episodes: Agent Workflow Memory (Wang et al., 2025b) induces reusable task workflows from past experience for transfer across structurally similar future tasks. *RAG-based personalization* adapts the retrieval pipeline itself to user preferences (Li et al., 2025b), with frameworks such as RAG-Anything (Guo et al., 2025b) providing flexible multimodal retrieval backbones. MemGPT (Packer et al., 2023) offers a complementary perspective, managing unbounded context through OS-inspired memory paging. More recently, hierarchical memory architectures have emerged: HiMem (Zhang et al., 2026) segments long-horizon dialogues into episode and note memories with conflict-aware reconsolidation, while G-Memory (Zhang et al., 2025) organizes multi-agent collaboration traces into a three-tier graph hierarchy of insights, queries, and interactions, achieving up to 20% gains in embodied-action benchmarks. These approaches provide useful building blocks. For tutoring settings, it is also useful to organize traces in domain-specific, typed, and semantically searchable forms across multiple resolutions (sessions, plan steps, tool invocations), so that downstream agents can reuse them for both explanation and question generation.

A.3. Knowledge Tracing and Intelligent Tutoring Systems

The need for structured learner modeling has a long history in education. Classical *knowledge tracing* (KT) estimates per-skill mastery probabilities from interaction logs: Bayesian Knowledge Tracing (Corbett and Anderson, 1995) uses two-state hidden Markov models, Deep Knowledge Tracing (Piech et al., 2015) replaces hand-crafted features with recurrent networks, DKVMN (Zhang et al., 2017) separates concept keys from evolving mastery values in a key-value memory, and context-aware attention models (Ghosh et al., 2020) further improve interpretability through monotonic attention over learner histories. Early *Intelligent Tutoring Systems* (ITS) such as Cognitive Tutor (Anderson et al.,

1995) were already providing step-level guidance through domain-specific production rules, enabling fine-grained diagnosis of learner actions at the cost of substantial manual domain engineering. Recent work points to a growing shift toward LLM-based tutoring, which brings broad domain coverage and flexible natural-language interaction while often placing less emphasis on explicit, inspectable learner models of the kind common in classical ITS (Chu et al., 2025; L  tourneau et al., 2025).

A.4. LLM-Based Educational Agents

Building on this shift, recent work has explored LLMs in dedicated tutoring roles along several complementary directions. Park et al. (2024) develop a conversational system with explicit student modeling, showing that tracking student state improves learning outcomes. Wang et al. (2025a) decompose the tutoring task across specialized agents for curriculum planning, dialogue, and assessment. Frankford et al. (2024) apply LLM tutoring to software engineering education, suggesting that richer student models can support higher-quality feedback. Open TutorAI (Hajji et al., 2026) provides an open-source platform with customizable avatars for multimodal interaction, while ParLD (Yao et al., 2026) introduces a preview–analyze–reason pipeline for conversational learning diagnosis via multi-agent collaboration. TutorLLM (Li et al., 2025c) couples knowledge tracing with RAG-based recommendation, and Dinucu-Jianu et al. (2025) align tutoring LLMs with pedagogical principles via reinforcement learning from simulated student interactions. Taken together, this literature now spans feedback generation, curriculum design, adaptive assessment, pedagogically aligned training, and open-ended evaluation (Chu et al., 2025; Macina et al., 2025). Across these systems, problem tutoring and question generation are often treated as separate components, with limited sharing of learner representations or performance signals. This design space is related to the bidirectional task coupling described in §2.4. At the same time, the literature remains broad in scope, and several closely related systems do not yet offer end-to-end reproducible releases for our exact tutoring setting.

B. Implementation Details of DeepTutor System

This appendix summarizes the two system-level loops behind DEEPTUTOR: the closed-loop tutoring cycle (§B.1) and the TutorBot autonomous agent loop (§B.2).

B.1. Closed-Loop Personalized Tutoring

Algorithm 1 gives the session-level execution order of DEEPTUTOR’s closed tutoring cycle. Starting from an initial learner profile $\mathcal{D}^{(0)}$ and an empty trace forest $\mathcal{F}^{(0)}$, each new session first assembles a personalized memory context C_{mem} from prior interactions and profile excerpts. The algorithm then branches according to the task type: problem tutoring uses retrieval-grounded investigation, planning, step-by-step tool use, and iterative answer writing, while question generation uses personalized idea filtering followed by critic-guided question, answer, and explanation construction. In both branches, the completed interaction is converted into a trace tree and fed back into the memory update, so subsequent sessions can reuse both high-level profile summaries and fine-grained interaction evidence.

B.2. TutorBot Autonomous Agent Loop

Algorithm 2 summarizes the autonomous agent loop used by TUTORBOT, the multi-channel deployment layer described in §3.3. Each inbound channel message triggers a three-phase cycle: context assembly, tool-augmented reasoning, and memory persistence. The bot first composes persona, long-term memory, active skills, and recent history into a working context; it then runs a bounded

Algorithm 1 Closed-Loop Personalized Tutoring

Require: learner profile $\mathcal{D}^{(0)}$; trace forest $\mathcal{F}^{(0)} = \emptyset$

```

1: for session  $j = 1, 2, \dots$  do
2:    $C_{\text{mem}} \leftarrow \text{PROFILEINJECT}(\mathcal{D}^{(j-1)}, \mathcal{F}^{(j-1)}, \text{task}_j)$ 
3:   if  $\text{task}_j = (\text{SOLVE}, q)$  then
4:      $C_{\text{rag}} \leftarrow \text{RAGTOOL}(q)$ 
5:      $\mathcal{E}_{\text{plan}} \leftarrow \text{INVESTIGATE}(q, C_{\text{rag}}, C_{\text{mem}}, \mathcal{F}^{(j-1)})$  ▷ ①
6:      $\mathcal{P} = \langle s_1, \dots, s_K \rangle \leftarrow \text{PLAN}(q, \mathcal{E}_{\text{plan}}, C_{\text{mem}})$ 
7:     for  $k = 1$  to  $K$  do ▷ ②
8:       while  $s_k$  not resolved do
9:          $(\alpha_t, \mathbf{s}_t) \leftarrow \text{TOOLAGENT}(q, \mathcal{P}, \mathcal{S}, C_{\text{rag}}, C_{\text{mem}})$ 
10:         $\mathcal{S} \leftarrow \mathcal{S} \cup \{\mathbf{s}_t\}$ 
11:        if  $\alpha_t = \text{REPLAN}$  then
12:           $\mathcal{P} \leftarrow \text{REVISE}(\mathcal{P}, \mathcal{S})$ 
13:        end if
14:      end while
15:       $\mathcal{S} \leftarrow \text{COMPRESS}(\mathcal{S}, k)$ 
16:    end for
17:     $a \leftarrow \text{ITERATIVEWRITE}(\mathcal{S}, C_{\text{mem}})$  ▷ ③
18:     $T_j \leftarrow \text{BUILDTRACE}(q, a)$ 
19:  else ▷  $\text{task}_j = (\text{GENERATE}, \tau)$ 
20:     $C_{\text{rag}} \leftarrow \text{RAGTOOL}(\tau)$ 
21:     $f_{\text{idea}} \leftarrow \emptyset$ 
22:    repeat
23:       $\mathcal{I} \leftarrow \text{IDEAAGENT}(\tau, C_{\text{rag}}, C_{\text{mem}}, f_{\text{idea}})$  ▷ ④
24:       $(\text{continue}, f_{\text{idea}}, \mathcal{T}_{1..N}) \leftarrow \text{EVALUATEIDEAS}(\mathcal{I}, \tau, C_{\text{mem}})$ 
25:    until  $\neg \text{continue}$ 
26:    for  $i = 1$  to  $N$  do ▷ ⑤
27:      repeat
28:         $(q_i, a_i, e_i) \leftarrow \text{GENERATOR}(\mathcal{T}_i, C_{\text{rag}}, C_{\text{mem}})$ 
29:         $(\text{pass}, f_i) \leftarrow \text{VALIDATOR}(q_i, a_i, e_i, \mathcal{T}_i)$ 
30:        if  $\neg \text{pass}$  then  $\mathcal{T}_i \leftarrow \text{REVISE}(\mathcal{T}_i, f_i)$ 
31:      end if
32:    until  $\text{pass}$ 
33:    end for
34:     $T_j \leftarrow \text{BUILDTRACE}(\tau, \{(q_i, a_i, e_i)\}_{i=1}^N)$ 
35:  end if
36:   $\mathcal{F}^{(j)} \leftarrow \mathcal{F}^{(j-1)} \cup \{T_j\}$ 
37:   $\mathcal{D}^{(j)} \leftarrow \text{MEMORYUPDATE}(\mathcal{D}^{(j-1)}, T_j)$ 
38: end for

```

ReAct-style tool-calling loop; finally, it persists the turn and consolidates older messages into durable memory when the session approaches the context budget.

The BUILDCONTEXT procedure concatenates the bot’s identity, runtime metadata, persona specification, user preferences, memory summaries, and active skill descriptions. The CONSOLIDATE procedure uses an LLM-based memory consolidator to append a timestamped history entry while updating the long-term learner profile. The concrete prompt templates and tool schemas are released with the repository rather than enumerated here.

Algorithm 2 TutorBot Autonomous Agent Loop

Require: message bus $\mathcal{B}$; LLM provider $\mathcal{M}$; tool registry $\mathcal{T}$; session store $\mathcal{S}$; context window W ; max iterations $I_{\max}$

```

1: loop ▷ Main event loop
2:    $m \leftarrow \mathcal{B}.\text{CONSUMEINBOUND}()$  ▷ Block until next message
3:    $\sigma \leftarrow \mathcal{S}.\text{GETORCREATE}(m.\text{session\_key})$  ▷ Phase 1: Context Assembly

4:    $\text{sys} \leftarrow \text{BUILDCONTEXT}(\sigma.\text{soul}, \sigma.\text{memory}, \sigma.\text{skills})$ 
5:    $H \leftarrow \sigma.\text{GETHISTORY}()$  ▷ Unconsolidated messages
6:    $\text{msgs} \leftarrow [\text{sys}] \parallel H \parallel [\text{WRAP}(m)]$  ▷ Phase 2: ReAct Tool-Calling Loop

7:   for  $i = 1$  to  $I_{\max}$  do
8:      $r \leftarrow \mathcal{M}.\text{CHAT}(\text{msgs}, \mathcal{T}.\text{DEFINITIONS}())$ 
9:     if  $r$  has no tool calls then
10:       $\text{response} \leftarrow r.\text{content}$  ▷ Final answer
11:      break
12:    end if
13:    Append assistant message (with tool calls) to  $\text{msgs}$ 
14:    for all tool call  $\tau$  in  $r.\text{tool\_calls}$  do
15:       $\text{result} \leftarrow \mathcal{T}.\text{EXECUTE}(\tau.\text{name}, \tau.\text{args})$ 
16:      Append tool result to  $\text{msgs}$ 
17:    end for
18:  end for ▷ Phase 3: Persist & Consolidate

19:   $\sigma.\text{APPENDTURN}(\text{msgs})$ 
20:  if  $\text{ESTIMATETOKENS}(\sigma) > W$  then ▷ Context pressure
21:     $\text{chunk} \leftarrow \sigma.\text{messages}[\sigma.\text{ptr} : \text{boundary}]$ 
22:     $\text{CONSOLIDATE}(\text{chunk}, \sigma.\text{memory})$  ▷ LLM-based distillation
23:     $\sigma.\text{ptr} \leftarrow \text{boundary}$ 
24:  end if
25:   $\mathcal{S}.\text{SAVE}(\sigma)$ 
26:   $\mathcal{B}.\text{PUBLISHOUTBOUND}(\text{response}, m.\text{channel})$ 
27: end loop

```

C. TutorBench Construction Details

This appendix supplements §4 with full construction details. Each benchmark *entry* combines a learner profile, a set of knowledge gaps, an interactive task, and the supporting source content. Below we describe the source PDF inventory (§C.1), the rejection-sampling procedure that produces high-quality tasks (§C.2), and the relationship between Trace Forest and benchmark-time simulation (§C.3).

C.1. PDF Source Inventory

TutorBench is built from 30 KBs derived from source PDFs. For textbook sources, each book is partitioned into three chapter-contiguous PDFs before KB construction; for research sources, each paper forms a single KB. The resulting PDF inventory is listed below.

Textbook-derived PDFs.

- All textbook PDFs are sourced from OpenStax.
- *Calculus Volume 2* (OpenStax, <https://openstax.org/details/books/calculus-volume-2>)

e-2/): Chapters 1–2, Chapters 3–4, and Chapters 5–6.

- *Calculus Volume 3* (OpenStax, <https://openstax.org/details/books/calculus-volume-3/>): Chapters 1–2, Chapters 3–4, and Chapters 5–6.
- *Principles of Economics 3e* (OpenStax, <https://openstax.org/details/books/principles-of-economics-3e/>): Chapters 1–6, Chapters 7–12, and Chapters 13–18.
- *Foundations of Information Systems* (OpenStax, <https://openstax.org/details/books/foundations-of-information-systems/>): Chapters 1–3, Chapters 4–6, and Chapters 7–9.
- *Introduction to Computer Science* (OpenStax, <https://openstax.org/details/books/introduction-computer-science/>): Chapters 1–3, Chapters 4–6, and Chapters 7–9.
- *Introduction to Philosophy* (OpenStax, <https://openstax.org/details/books/introduction-philosophy/>): Chapters 1–4, Chapters 5–8, and Chapters 9–12.
- *Introduction to Business* (OpenStax, <https://openstax.org/details/books/introduction-business/>): Chapters 1–5, Chapters 6–10, and Chapters 11–15.
- *Writing Guide with Handbook* (OpenStax, <https://openstax.org/details/books/writing-guide/>): Chapters 1–6, Chapters 7–12, and Chapters 13–18.

Paper-derived PDFs.

- *Memory in the Age of AI Agents* (Hu et al., 2025).
- *DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning* (Guo et al., 2025a).
- *LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code* (Jain et al., 2025).
- *OpenVLA: An Open-Source Vision-Language-Action Model* (Kim et al., 2024).
- *Towards Reasoning Era: A Survey of Long Chain-of-Thought for Reasoning Large Language Models* (Chen et al., 2025).
- *YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information* (Wang et al., 2024b).

C.2. Task Generation and Rejection Sampling

Given a profile p and its associated knowledge gaps G , the task generator creates interactive learning tasks that naturally expose one or more gaps during a tutoring conversation. Four task types are supported—*concept understanding* (30%), *problem solving* (30%), *application* (20%), and *comparison* (20%)—with sampling weights reflecting typical student learning request distributions. A batch rejection sampler then filters out tasks whose gaps are incoherent, misaligned with the task description, or conversationally unnatural / internally inconsistent, and regenerates until the minimum task count is satisfied. Algorithm 3 gives the full procedure.

The REJECTSAMPLE oracle verifies three criteria:

1. **Gap coherence.** The gaps targeted by the task must be logically related and form a coherent learning topic; tasks requiring the student to jump between disconnected concepts are rejected.
2. **Task-gap fit.** The task description, initial student message, and success criteria must organically connect to the targeted gaps—pursuing the task should naturally expose those gaps without artificial steering.
3. **Conversational naturalness and consistency.** The resulting task should read like a plausible learner request for the given profile, rather than an artifact assembled solely to surface benchmark gaps. Items whose gaps contradict one another or conflict with the task description are rejected under this criterion.

The oracle is instructed to apply strict criteria: when in doubt, reject. This conservative stance accepts

Algorithm 3 Task Generation with Rejection Sampling**Require:** Profile p , gaps G , knowledge scope scope , source pages S ; minimum tasks $T_{\min} = 3$

```

1:  $\text{accepted} \leftarrow \emptyset$ 
2: repeat
3:    $\mathcal{T}^{\text{cand}} \leftarrow \text{GENERATE\_TASKS}(p, G, \text{scope})$  ▷ batch LLM generation
4:   for each candidate task  $t \in \mathcal{T}^{\text{cand}}$  do
5:      $\text{ok} \leftarrow \text{REJECTSAMPLE}(t, G, S)$  ▷ 3-criterion check (below)
6:     if  $\text{ok}$  then
7:        $\text{accepted} \leftarrow \text{accepted} \cup \{t\}$ 
8:     end if
9:   end for
10: until  $|\text{accepted}| \geq T_{\min}$ 
11: return  $\text{accepted}$ 

```

a higher regeneration rate in exchange for higher task quality, a trade-off justified by the one-time offline nature of benchmark construction. Each accepted task carries a structured record: task type, title, description, initial student message (matched to the profile’s personality), targeted gap IDs, expected gap exposure trajectory, success criteria, and difficulty rating. Step 1 then packages the profile, selected gaps, accepted task, and the corresponding sampled `source_content` into one benchmark entry, which becomes the unit consumed by the simulation and evaluation pipeline.

C.3. Trace Forest and Benchmark Runtime Memory

The benchmark entry files produced during construction do *not* serialize a Trace Forest. Trace Forest is a *runtime* memory mechanism: during Step 2 simulation, DEEPTUTOR backends with cross-session memory enabled build and query a Trace Forest to condition both problem-tutoring responses and practice-question generation. Ablated variants disable this memory while keeping the underlying benchmark entries unchanged. The key implication for reproducibility is that the benchmark fixes the same entry objects for all systems; only the DEEPTUTOR family differs in whether it reads from and writes to this runtime memory during multi-session simulation.

D. Evaluation Protocol and Hyperparameters

This section provides full evaluation details for the interactive evaluation (§D.1), the human preference alignment study (§D.3), and the general problem-solving benchmarks (§D.4), together with baseline definitions (§D.2), hyperparameters (§D.5), and implementation specifics (§D.6).

D.1. First-Person Interactive Evaluation Details

Figure 4 in the main paper summarizes the full interactive evaluation loop used for TutorBench; this appendix provides the implementation details. Each evaluation begins from a benchmark entry containing a learner profile, diagnosed knowledge gaps, an interactive task, and relevant source materials. The student simulator turns these inputs into first-person beliefs and maintains a dynamic tutoring list during the dialogue. At each turn, it either asks the tutor for help on an unfinished tutoring item or, once the tutoring phase is complete, asks the tutor to generate a customized quiz. All tutor responses are recorded as traces and later judged with personalized rubrics derived from the same learner state and task context.

D.1.1. Student Simulator

The student simulator is an LLM agent initialized from a TutorBench entry (profile + gaps + task). Each gap is pre-transformed into a first-person belief statement so the simulator *acts as* the student rather than narrating errors from an outside view. The system prompt below governs simulator behavior throughout the interaction; the template variables are filled from the entry at initialization time.

```
You are role-playing as a student seeking help from an AI tutor.
Stay in character throughout the entire conversation. Never break
character.

# Who You Are
{personality}

# Your Background
{education_background}

# Why You Are Here
{learning_purpose}

# What You Know
You are confident about these topics:
{known_well}

You have vague or partial understanding (you might get details wrong):
{partially_known}

You have NO knowledge of the following (never encountered):
{unknown}

# What You Believe (IMPORTANT -- these feel true to you)
{beliefs}

# Behavioral Rules
1. NEVER display knowledge listed as "unknown". If the tutor asks
   about something you don't know, say so honestly.
2. When the tutor teaches something new, try to rephrase it in your
   own words. It's okay to rephrase imperfectly.
3. If the tutor asks "do you understand?", be honest based on whether
   the explanation actually addressed your confusion.
4. Keep responses concise: 1-4 sentences typically.
5. Stay consistent with your personality throughout.
6. You may ask follow-up questions or request examples.
7. When you feel you've understood, PREFER asking for a practice
   problem before ending -- testing yourself helps solidify learning.

# Ending the conversation (ONLY when done)
Use [ACTION: task_complete] ONLY if ALL conditions are true:
- You are explicitly done.
- You have zero remaining questions.
- You are NOT requesting anything else.
- Your message is a natural closing/goodbye.

CRITICAL: If your message contains any question, uncertainty, or
request for more help, do NOT use [ACTION: task_complete].
```

Listing 1 | Student Simulator System Prompt

Dynamic gap pacing. The {beliefs} placeholder is populated with first-person formulations of each knowledge gap (e.g., “I think the Poynting vector always points in the direction of wave propagation”). The simulator’s behavioral rules enforce realistic resistance patterns: the student may defend misconceptions, partially accept corrections, and only signal completion after sufficient scaffolded explanation. This prevents trivially short sessions where the student immediately agrees with any tutor statement.

D.1.2. Evaluation Rubrics

Tutoring quality is assessed along ten dimensions, each scored on a 1–5 Likert scale by an LLM judge. The ten metrics are organized into two groups—five **tutoring-side** metrics that evaluate every tutor turn during the dialogue, and five **practice-side** metrics that score the generated questions after each session. In addition, the evaluator records raw turn counts as a non-LLM structural statistic. The pipeline reports all metrics independently; no weighted composite score is computed. The dimensions operationalize constructs from source-grounded generation, learner modeling and formative feedback, multiple representations for explanation, and educational question assessment (Ainsworth, 2006; Corbett and Anderson, 1995; Kurdi et al., 2020; Maynez et al., 2020; Shute, 2008).

Tutoring-Side Metrics (turn-level, 1–5 each). Each metric below is evaluated on every student-to-tutor turn pair. When practice questions are present, the final dialogue turn that emits them is excluded from tutoring-side scoring to keep the tutoring and question-generation phases separate.

- **Source Faithfulness (SF)** captures factual consistency with the source material provided in each benchmark entry, following work on faithful and attributable generation (Maynez et al., 2020; Rashkin et al., 2023). The judge examines four sub-dimensions: (a) *factual consistency*—whether tutor claims agree with the source rather than contradicting it; (b) *terminological consistency*—whether the tutor follows the source’s definitions and framing; (c) *source grounding*—whether the response shows evidence of drawing on source-specific content, not just generic knowledge; and (d) *source attribution*—whether the tutor explicitly attributes information via citation markers or natural-language references. Pedagogical elaboration beyond the source—analogy, worked examples, motivating applications—is explicitly *not* penalized, as long as it does not distort the source. A score of 5 requires accuracy, consistent terminology, clear source grounding, and explicit attribution; a score of 1 indicates core claims that are wrong or that directly conflict with the source.
- **Personalization (PER)** reflects how specifically the tutor addresses the learner’s current confusion while remaining aligned with the learner state developed over the session (Corbett and Anderson, 1995; Vygotsky, 1978). The judge checks whether the response is tailored to the student’s profile, the misconceptions surfaced in prior turns, and the particular question just asked—rather than offering a generic textbook answer. A score of 5 requires strong, visible tailoring to this specific learner’s misunderstanding and conversational trajectory; a score of 3 is partially tailored but still generic in key parts; a score of 1 indicates a fully generic response that ignores learner-specific context entirely.
- **Applicability (APP)** evaluates whether the guidance is concrete and immediately actionable for the student (Hattie and Timperley, 2007; Shute, 2008). High scores require the tutor to provide specific, executable content such as worked examples, step-by-step procedures, verifiable calculations, or diagnostic questions that let the student make immediate progress on their problem. A score of 5 means the student can directly act on the instruction; a score of 1 means the response is vague, abstract, or impossible to verify.
- **Vividness (VID)** rewards representationally rich, engaging presentation that goes beyond monolithic plain-text prose (Ainsworth, 2006; Mayer, 2003). The judge assesses modality diversity: effective use of structured lists, equations paired with intuitive explanations, comparison tables, code demonstrations, or other representational forms that genuinely aid understanding. A score of 5 indicates rich multimodal delivery whose format choices clearly enhance learning; a score of 1 indicates flat, undifferentiated text with no meaningful modality variation.
- **Logical Depth (LD)** assesses the presence of explicit multi-step causal or analytical reasoning, as opposed to bare assertions (Chi et al., 1989; Koedinger et al., 2012). The judge looks for coherent reasoning chains: does the tutor explain *why* a result holds, trace cause-and-effect relationships, or

walk the student through intermediate steps? A score of 5 requires clear, well-structured multi-step reasoning; a score of 1 means the tutor provides conclusions or facts without any substantive justification.

Practice-Side Metrics (session-level, 1–5 each). After the main tutoring dialogue concludes, the tutor generates five practice multiple-choice questions (MCQs). Each question is evaluated independently across the following five dimensions:

- **Fitness (FIT)** measures alignment with the session-level diagnosed weaknesses at appropriate difficulty (Lord, 1980; Wood et al., 1976). The judge checks whether the question targets the knowledge gaps surfaced during the dialogue, matches the student’s proficiency level, and is neither trivially easy nor unreasonably hard given the learner profile. A score of 5 means the question precisely targets the student’s weak points at a suitable challenge level; a score of 1 means the question is irrelevant to the diagnosed gaps or grossly miscalibrated in difficulty.
- **Groundedness (GND)** requires factual anchoring in the source material, mirroring the structure of Source Faithfulness on the solve side (Gao et al., 2023; Lewis et al., 2020). The judge checks four parallel sub-dimensions: factual consistency, terminological consistency, source grounding, and source attribution of the question stem, correct answer, and explanation. A score of 5 means the question content is fully consistent with and clearly derived from the source; a score of 1 means the question uses fabricated claims or definitions that conflict with the source.
- **Diversity (DIV)** rewards novelty in angle and cognitive demand (Anderson et al., 2001; Kurdi et al., 2020). The judge evaluates whether the question presents a distinct perspective, question format, or cognitive action (e.g., application, analysis, comparison) compared with typical rote recall items. A score of 5 reflects a creative, non-templated question that engages higher-order thinking; a score of 1 indicates a standard fill-in-the-blank or direct-recall item indistinguishable from routine exercises.
- **Answer Quality (ANS)** evaluates the correctness of the reference answer and the plausibility of distractors (Haladyna et al., 2002). The judge verifies that the correct option is unambiguously right, that the explanation is clear and consistent with the chosen answer, and that distractors are wrong yet plausible enough to be pedagogically informative. A score of 5 requires a flawless key, a lucid explanation, and well-crafted distractors; a score of 1 indicates an incorrect key, a contradictory explanation, or nonsensical options.
- **Cross Concept (CC)** tests whether the question meaningfully integrates multiple concepts surfaced throughout the session, rather than testing a single isolated fact (Barnett and Ceci, 2002; Chi et al., 1981). The judge checks whether the question requires the student to synthesize ideas from different parts of the material or to reason about relationships between concepts. A score of 5 means the question demands genuine cross-concept reasoning; a score of 1 means it tests only one narrow, isolated point.

Auxiliary statistics. Beyond the ten judge-scored metrics, the evaluator records raw turn counts (student turns, tutor turns, and paired turns) as a structural measure of dialogue length. These counts are reported directly and are not normalized to the 1–5 scale. Additionally, the evaluator tracks judge reliability statistics such as parse-failure rates per metric; these are used to monitor evaluation stability but are not part of the quality scores themselves.

D.1.3. Protocol Details

The full benchmark pipeline runs in three stages. **Step 1** (entry generation): TutorBench entries are generated offline using the pipeline described in Appendix C. **Step 2** (multi-session simulation): for each KB/profile/backend tuple, the student simulator is initialized from the entry set and runs up to

`max_turns` = 30 student turns per session in the default batch pipeline; transcripts are saved together with five practice questions produced at session end. **Step 3** (LLM-as-judge): each saved transcript is scored for source faithfulness, four teaching-quality dimensions, turn counts, and practice-question quality.

For the reported interactive evaluation, *Gemini-3-Flash* powers both the student simulator and all tutor backbones, including DEEPTUTOR, while *Claude Sonnet 4.6* serves as the fixed judge with temperature = 0.0. To mitigate judge variance, each transcript is scored three times and the scores are averaged.

All 1–5 scores are averaged to four decimal places at session, multi-session, and backend-summary levels. The current pipeline does *not* compute a weighted overall benchmark score; instead it reports the independent metrics directly.

D.1.4. Cross-Domain Metric Table

Table 4 reports the complete domain-by-metric breakdown for the first-person interactive simulation visualized in Figure 5.

Domain	Tutoring Quality						Practice Quality						
	SF	PER	APP	VID	LD	Avg	FIT	GND	DIV	ANS	CC	Avg	OQ
Humanities	3.24	4.55	4.45	4.75	4.71	4.34	3.15	2.80	3.58	3.82	3.32	3.33	3.84
Sciences	3.56	4.64	4.63	4.80	4.67	4.46	3.43	3.03	3.16	4.14	3.53	3.46	3.96
Engineering	3.59	4.54	4.48	4.66	4.29	4.31	3.24	3.05	3.53	4.04	3.47	3.46	3.89
Business	3.53	4.56	4.74	4.93	4.73	4.50	3.54	2.92	3.42	4.16	3.39	3.48	3.99
Research	2.87	4.64	4.52	4.92	4.64	4.32	3.39	3.02	3.52	3.75	3.21	3.38	3.85

Table 4 | Full cross-domain results for the first-person interactive simulation. The main paper visualizes the same data in Figure 5. OQ is the average over the five tutoring-side and five practice-side metrics.

D.2. Baseline and Variant Definitions

We compare DEEPTUTOR against four prompt-based tutor baselines and three ablation variants, all running within a single unified simulation harness. As discussed in Appendix A.4, existing educational and agentic systems vary widely in scope and availability; we therefore design baselines that isolate specific reasoning strategies while sharing the same backbone (*Gemini-3-Flash*) and KB access as DEEPTUTOR. Below we give the algorithmic definition of each baseline, followed by a brief note on the three ablation variants (w/o SKG, w/o DPM, w/o SKG + DPM) that selectively disable components of the full pipeline.

Notation. All baseline algorithms below share the following primitives. $\text{RAG}(m, \mathcal{K}, \text{mode}, k)$ retrieves k passages from knowledge base $\mathcal{K}$ for query m ; $\text{CONCAT}(\cdot)$ concatenates its arguments into a single context string; and $\text{LLM}(p, H, \text{ctx})$ denotes one LLM generation call with system prompt p , conversation history H , and user-side context ctx . Prompt variables $p.$ are symbolic shorthands; their concrete prompt templates are included in the released repository.

Naive Tutor (mock). The simplest baseline: a single LLM call with a minimal tutoring prompt p_{tutor} (instructing the model to act as a helpful and patient tutor), the conversation history, and a naive-mode retrieved KB snippet prepended to the current student message. There is no planning, no

tool-execution loop, and no cross-session memory.

Algorithm 4 Naive Tutor (Mock)

Require: Student message m , conversation history H , knowledge base $\mathcal{K}$

Prompt: p_{tutor} (tutoring persona)

- 1: $r_{\text{rag}} \leftarrow \text{RAG}(m, \mathcal{K}, \text{mode} = \text{naive}, k=2)$
 - 2: $\text{ctx} \leftarrow \text{CONCAT}(r_{\text{rag}}, m)$
 - 3: $\text{response} \leftarrow \text{LLM}(p_{\text{tutor}}, H, \text{ctx})$
 - 4: **return** response
-

CoT Tutor (cot). Identical to the Naive Tutor except that the system prompt p_{cot} extends p_{tutor} with a chain-of-thought directive (Wei et al., 2022) (“think step by step before answering”), asking the model to reason internally before producing its tutoring response. The implementation still uses a single generation call.

Algorithm 5 CoT Tutor

Require: Student message m , conversation history H , knowledge base $\mathcal{K}$

Prompt: p_{cot} (tutoring persona + chain-of-thought directive)

- 1: $r_{\text{rag}} \leftarrow \text{RAG}(m, \mathcal{K}, \text{mode} = \text{naive}, k=2)$
 - 2: $\text{ctx} \leftarrow \text{CONCAT}(r_{\text{rag}}, m)$
 - 3: $\text{response} \leftarrow \text{LLM}(p_{\text{cot}}, H, \text{ctx})$
 - 4: **return** response
-

Self-Refine Tutor (self_refine). Inspired by Madaan et al. (2023), this baseline uses two sequential LLM calls. The first call uses p_{tutor} (the same tutoring persona as mock) to produce an initial draft; the second call uses p_{ref} , which instructs a pedagogical quality reviewer to refine the draft for clarity, specificity, and pedagogical effectiveness. The refinement pass receives the draft together with the current student message, conversation history, and the same retrieved context.

Algorithm 6 Self-Refine Tutor

Require: Student message m , conversation history H , knowledge base $\mathcal{K}$

Prompts: p_{tutor} (tutoring persona), p_{ref} (pedagogical reviewer)

- 1: $r_{\text{rag}} \leftarrow \text{RAG}(m, \mathcal{K}, \text{mode} = \text{naive}, k=2)$
 - 2: $\text{ctx} \leftarrow \text{CONCAT}(r_{\text{rag}}, m)$
 - 3: $\text{draft} \leftarrow \text{LLM}(p_{\text{tutor}}, H, \text{ctx})$ ▷ generate initial response
 - 4: $\text{ctx}' \leftarrow \text{CONCAT}(r_{\text{rag}}, m, \text{draft})$
 - 5: $\text{response} \leftarrow \text{LLM}(p_{\text{ref}}, H, \text{ctx}')$ ▷ refine for clarity & pedagogy
 - 6: **return** response
-

ReAct Tutor (react). This baseline implements a *single-round* ReAct-inspired loop (Yao et al., 2023) per student turn via four sequential LLM calls, each with a dedicated prompt: p_{think} analyses the student’s question and diagnoses what they most need next; p_{act} decides on a concrete tutoring action plan; p_{obs} reviews the thought–action pair and notes any adjustment; and p_{tutor} (the shared tutoring persona) synthesises the accumulated reasoning into a final student-facing response. All four calls operate over the same retrieved context c_0 and conversation history. Relative to DEEPTUTOR, this baseline does not include a multi-step planner, external tool execution beyond the shared naive RAG context, or cross-session memory.

Baseline and ablation metric breakdown. Table 5 provides the full per-metric breakdown across the released baselines and DEEPTUTOR variants.

Algorithm 7 ReAct Tutor (single-round per turn)**Require:** Student message m , history H , knowledge base $\mathcal{K}$ **Prompts:** p_{think} , p_{act} , p_{obs} , p_{tutor} 1: $r \leftarrow \text{RAG}(m, \mathcal{K}, \text{naive}, k=2)$ 2: $c_0 \leftarrow \text{CONCAT}(r, H, m)$ 3: $th \leftarrow \text{LLM}(p_{\text{think}}, c_0)$ 4: $act \leftarrow \text{LLM}(p_{\text{act}}, \text{CONCAT}(c_0, th))$ 5: $obs \leftarrow \text{LLM}(p_{\text{obs}}, \text{CONCAT}(c_0, th, act))$ 6: $resp \leftarrow \text{LLM}(p_{\text{tutor}}, H, \text{CONCAT}(c_0, th, act, obs))$ 7: **return** $resp$

▷ base context
 ▷ diagnose need
 ▷ plan action
 ▷ review

Variant	SF↑	PER↑	APP↑	VID↑	LD↑	FIT↑	GND↑	DIV↑	ANS↑	CC↑
Naive Tutor	3.2192	4.2406	4.4438	3.8895	3.9894	3.1978	2.4563	2.8267	3.8744	3.1200
CoT Tutor	3.3395	4.2238	4.4709	3.8190	4.0096	3.1800	2.4978	2.7911	3.8120	3.0444
Self-Refine Tutor	3.2785	4.2756	4.6090	3.9384	4.1414	3.1711	2.4911	2.8756	3.8844	2.9689
ReAct Tutor	3.3472	4.3722	4.4023	3.7044	3.9598	3.1578	2.4667	2.7511	<u>3.9444</u>	3.0667
DEEPTUTOR	3.3641	4.5930	4.5638	4.8145	4.6079	3.3500	2.9600	3.4422	3.9767	3.3822
w/o SKG	3.1057	<u>4.4091</u>	4.5537	4.8316	<u>4.5348</u>	<u>3.3055</u>	2.2311	3.3022	3.8355	3.1911
w/o DPM	<u>3.3554</u>	4.2243	4.5695	<u>4.8269</u>	4.5156	3.1538	<u>2.8022</u>	<u>3.3400</u>	3.9011	<u>3.3089</u>
w/o SKG + DPM	3.1609	4.1724	<u>4.5983</u>	4.8200	4.4811	3.2533	2.2211	3.2733	3.8522	3.1356

Table 5 | Full metric breakdown across the released baselines and DEEPTUTOR variants. All columns are 1–5 averages, filled from the released overall evaluation summary manifest. Bold indicates the best value in each column; underline indicates the second-best value. Abbreviations: SF = source faithfulness; PER = personalization; APP = applicability; VID = vividness; LD = logical depth; FIT = fitness; GND = groundedness; DIV = diversity; ANS = answer quality; CC = cross concept.

D.3. Human Preference Alignment Study

The human alignment study in §5.3 is designed to test whether the LLM judge’s relative system preferences agree with independent human judgments, rather than to measure downstream learning gains.

Sampling and materials. We use a domain-stratified subset of 45 TutorBench sessions, with nine sessions sampled from each of the five domains used in Table 4. For every selected session, we collect the full DEEPTUTOR and *Naive Tutor (Mock)* outputs generated under the same benchmark entry: identical learner profile, task description, source material, student-simulator protocol, and practice-generation request. The comparison packet shown to annotators contains the learner profile, task description, source material, tutoring transcript, and generated practice questions for both systems.

Human annotation. We recruit ten student annotators with relevant academic training: six undergraduate students and four graduate students (two master’s students and two Ph.D. students). Each domain is assigned to two annotators. Research sessions are evaluated by the two Ph.D. students; Engineering and Business each include one domain-related master’s student; the remaining domain assignments are completed by domain-related undergraduates. Each packet is evaluated by both assigned annotators in a blind A/B interface. System names are hidden, and the order of A and B is randomly flipped per session to reduce order bias. Annotators are given the same ten metric definitions used by the LLM judge: Source Faithfulness, Personalization, Applicability, Vividness, Logical Depth, Fitness, Groundedness, Diversity, Answer Quality, and Cross Concept. For each metric, the annotator selects one of three labels: A is better, B is better, or the two systems are indistinguishable. This pairwise design avoids the scale-calibration problem of absolute 1–5 human scoring while directly

Benchmark	Category	Size	Subset	Format
HLE (Phan et al., 2026)	STEM reasoning	~2,500	500 (text+mm)	open-ended
GPQA-Diamond (Rein et al., 2024)	STEM reasoning	198	all 198	4-choice MCQ
LiveBench (White et al., 2025)	reasoning	~200	reasoning cat.	mixed
GAIA (Mialon et al., 2024)	agentic	~165	validation (L1–L3)	open-ended
AA-LCR (Team, 2025)	long-context	100	all 100	open-ended

Table 6 | Public benchmarks used in the general problem-solving evaluation. “Size” refers to the total available test items; “Subset” is the number we evaluate on. mm = multimodal (text + images).

testing the system preference implied by the automatic judge.

Aggregation and LLM comparison. Human labels are first mapped back to system identities and then aggregated for each session–metric pair; when the two annotators do not select the same system preference, the aggregate label is treated as a tie. For the LLM side, we use the corresponding LLM-judge preference label for the same session and metric. Figure 7 reports, for each metric, the share of comparisons where humans prefer DEEPTUTOR, mark a tie, or prefer Mock, alongside the corresponding LLM-judge preference shares.

Alignment statistics. The correlation statistics reported in Figure 6 and §5.3 are computed over the ten metric-level DEEPTUTOR win-rate pairs, rather than over all individual session–metric labels. Pearson $r = 0.82$ ($p = 0.0038$) and Spearman $\rho = 0.83$ ($p = 0.0027$) therefore test whether the rubric dimensions most favored by human raters are also the dimensions most favored by the LLM judge. At the same metric-summary level, the directional winner agrees on all ten metrics: both human raters and the LLM judge assign the largest preference share to DEEPTUTOR for every metric.

D.4. General Problem-Solving Setup

D.4.1. Benchmark Descriptions

Table 6 summarizes the five public benchmarks used in §5.5.

D.4.2. Evaluation Protocol

Correctness is assessed via a two-stage pipeline. **Stage 1 (Extract):** an LLM extracts the model’s final answer from its free-form output (model: Claude Sonnet 4.6, temperature = 0.0, max tokens = 16384). **Stage 2 (Judge):** a second LLM call compares the extracted answer to the ground truth using benchmark-specific rubrics (same model and settings as Stage 1 for consistency). Benchmark-specific rubrics cover exact match for GAIA, option matching or text equivalence for GPQA and HLE, and textual equivalence checks for open-ended benchmarks such as LiveBench and AA-LCR. All Pass@1 scores are reported as percentages.

D.4.3. Pipeline Configuration

Table 7 lists the tool configuration and concurrency settings for the DEEPTUTOR pipeline on each benchmark. This is the same solver-only configuration described in §5.5: the Hybrid Personalization Engine is disabled, including both Static Knowledge Grounding (SKG) and Dynamic Personal Memory (DPM). Thus no course knowledge base, learner profile, or trace-forest retrieval is active in the public benchmark runs.

Benchmark	Tools Enabled	Max Tokens	Concurrency
HLE	code_execute, reason	16384	30
GPQA-D	code_execute, reason	16384	50
LiveBench	code_execute, reason	16384	10
GAIA	code_execute, reason, web_search	16384	20
AA-LCR	code_execute, reason	16384	25

Table 7 | Per-benchmark pipeline configuration. `done` and `replan` are always available. Temperature is 0.0 across all benchmarks.

D.5. Agent Hyperparameters

Table 8 reports the temperature and maximum token budget for each module. The solve module uses low temperature (0.3) for deterministic multi-step reasoning; the question module uses higher temperature (0.7) to promote ideation diversity; the personalization module uses an intermediate value (0.5) balancing consistency with adaptive profile updates.

Module	Agents	Temp.	Max Tokens
Solve	planner, solver, writer	0.3	8192
Question	idea, evaluator, generator, validator	0.7	4096
Personalization	summary, weakness, reflection	0.5	8192

Table 8 | Agent hyperparameters. All modules share a single set of parameters; per-agent overrides are not used in the reported experiments.

D.6. Implementation Details

For reproducibility, we detail key implementation choices not fully covered in the main text.

Weakness State Transitions. Weakness state transitions (active $\rightarrow$ resolved) are *evidence-gated* rather than governed by numeric confidence thresholds. The Weakness Agent uses prompt-based criteria: a weakness is marked resolved when the user answers correctly on the topic in recent sessions *and* stops re-asking about it. The benchmark evaluator enforces strict resolution criteria—explicit correction of the misconception, a clear explanation of why the old understanding was wrong, and scaffolded reasoning for complex concepts. Profile evolution moves resolved concepts into `known_well` and removes them from `unknown` and `partially_known`.

Trace Retrieval Budget. Trace retrieval uses a fixed `top_k` = 3 for semantic search across the trace forest. Level-specific filtering is applied by context: idea and evaluator agents query level-1 nodes (session roots); generator context queries level-3 nodes for wrong-answer patterns. The `SEARCHTRACE` tool defaults to `top_k` = 5, and individual trace detail views are truncated to 6,000 characters. Scratchpad context budgets are 6,000 tokens for the solver and 12,000 tokens for the writer. Conversation history in the benchmark simulator is truncated to 10,000 characters.

Embedding Models. Both the trace forest and RAG indexing share the same embedding service. The embedding model is configured via environment variables; supported options include OpenAI `text-embedding-3-large` (3,072 dimensions), Cohere `embed-v4.0` (1,024 dimensions), and Jina `jina-embeddings-v3` (1,024 dimensions).

Knowledge Graph Construction. Knowledge graph construction is delegated to LightRAG, which performs chunking, entity extraction, and embedding internally. RAGAnything adds MinerU-based PDF parsing (images, tables, equations) before insertion. Our implementation uses LightRAG’s default entity extraction pipeline without custom entity alignment or additional quality-control filtering.

Additional Solver Parameters. Table 9 lists additional parameters governing the solver-only and problem-tutoring pipelines.

Parameter	Value	Parameter	Value
Max ReAct iterations per step	5	RAG mode (planner)	hybrid
Max plan steps	10	Question idea loop rounds	3
Max replans	2	Ideas per round	5
Observation truncation	2,000 tokens	Generator max retries	2
Planner pre-retrieval queries	3	Question RAG mode	naive
RAG top_k (planner)	8	Memory agent max ReAct rounds	6

Table 9 | Additional pipeline parameters.

D.7. Backbone Models

Table 10 lists the five backbone families evaluated in the general problem-solving experiments (§5.5). All backbone results are obtained via OpenRouter-compatible APIs with greedy decoding (temperature = 0.0).

Backbone	Provider	Context Window
Gemini-3-Flash	Google DeepMind	1M tokens
Sonnet-4.5	Anthropic	200K tokens
Qwen-3.5-Plus	Alibaba Cloud	128K tokens
GPT-5-Mini	OpenAI	128K tokens
Minimax-M2.5	MiniMax	1M tokens

Table 10 | Backbone families used in the general problem-solving evaluation. Context window sizes are approximate as reported at evaluation time.